

PREMIER UNIVERSITY, CHATTOGRAM

Department of Computer Science & Engineering



Final Year Thesis Report

On

**Bengali Spam Email Detection Using Machine Learning  
And Deep Learning Model**

**SUBMITTED BY**

**Name:** Jannatul Ferdous

**ID:** 1803510201665

**Name:** Saima Sultana

**ID:** 1803510201689

**Name:** S M Tahnim Mahir

**ID:** 1803510201691

In partial fulfillment for the degree of  
Bachelor of Science in Computer Science & Engineering  
under the Supervision of

Dhrubajyoti Das

Lecturer

Department of Computer Science & Engineering

Premier University, Chattogram

April, 2024

---

# Author's Declaration of Originality

We hereby declare that the thesis work entitled “ **Bengali Spam Email Detection Using Machine Learning And Deep Learning Model** ” submitted to the Premier University, is a record of an original work done by us under the guidance of Mr. Dhrubajyoti Das, Lecturer, Department of Computer Science & Engineering, Premier University, Chattogram and this work is submitted for fulfillment of the degree of Bachelor Science in Computer Science & Engineering. We can assure that the result of this thesis has not been submitted to any other university.

---

Jannatul Ferdous  
ID: 1803510201665

---

Saima Sultana  
ID: 1803510201689

---

S M Tahnim Mahir  
ID: 1803510201691

---

# CERTIFICATION

The thesis entitled “ **Bengali Spam Email Detection Using Machine Learning And Deep Learning Model** ” submitted by, **Jannatul Ferdous**, ID: 1803510201665, **Saima Sultana**, ID: 1803510201689, **S M Tahnim Mahir**, ID: 1803510201691 has been accepted as satisfactory in partial fulfillment of the degree of Bachelor Science in Computer Science & Engineering (CSE) to be awarded by Premier University, Chattogram.

---

Dr. Shahid Md. Asif Iqbal  
Chairman & Associate Professor  
Department of Computer Science & Engineering  
Premier University, Chattogram

---

Dhrubajyoti Das  
Lecturer  
Department of Computer Science & Engineering  
Premier University, Chattogram

---

*It is my genuine gratefulness and warmest regard that  
I dedicate this work to my beloved  
**Father and Mother***

# Table of Contents

|                                                                   |            |
|-------------------------------------------------------------------|------------|
| <b>List of Figures</b>                                            | <b>vi</b>  |
| <b>List of Tables</b>                                             | <b>vii</b> |
| <b>1 Introduction</b>                                             | <b>1</b>   |
| 1.1 Introduction . . . . .                                        | 1          |
| 1.2 Motivation . . . . .                                          | 3          |
| 1.3 Objective . . . . .                                           | 3          |
| 1.4 Operating System and IDE . . . . .                            | 4          |
| 1.5 Application . . . . .                                         | 4          |
| 1.6 Summary . . . . .                                             | 4          |
| <b>2 Literature Review</b>                                        | <b>5</b>   |
| 2.1 Overview . . . . .                                            | 5          |
| 2.2 Literature Review . . . . .                                   | 5          |
| 2.3 Summary . . . . .                                             | 7          |
| <b>3 Background Study</b>                                         | <b>9</b>   |
| 3.1 Overview . . . . .                                            | 9          |
| 3.2 Type of Mails . . . . .                                       | 9          |
| 3.2.1 Spam Emails . . . . .                                       | 10         |
| 3.2.2 Ham Emails . . . . .                                        | 10         |
| 3.2.3 Promotional Emails . . . . .                                | 10         |
| 3.3 Significance of the work . . . . .                            | 11         |
| 3.4 Challenges of Bangla Spam Detection . . . . .                 | 11         |
| 3.5 Bangla spam email detection algorithms that we used . . . . . | 12         |
| 3.5.1 Machine Learning . . . . .                                  | 12         |
| 3.5.2 Deep Learning . . . . .                                     | 18         |
| <b>4 Methodology</b>                                              | <b>20</b>  |
| 4.1 Introduction . . . . .                                        | 20         |
| 4.2 Dataset Collection Procedure . . . . .                        | 21         |
| 4.3 Proposed Methodology . . . . .                                | 22         |
| 4.3.1 Data Preprocessing . . . . .                                | 22         |
| 4.3.2 Data Annotation . . . . .                                   | 23         |
| 4.3.3 Feature extraction . . . . .                                | 23         |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| 4.3.4    | Data Splitting . . . . .                                   | 24        |
| 4.4      | Working Procedure of Deep Learning . . . . .               | 25        |
| 4.4.1    | Architecture Of Our Model . . . . .                        | 25        |
| 4.4.2    | Embedding layer . . . . .                                  | 25        |
| 4.4.3    | Bidirectional LSTM . . . . .                               | 26        |
| 4.4.4    | Dropout layer . . . . .                                    | 27        |
| 4.4.5    | Flatten layer . . . . .                                    | 27        |
| 4.4.6    | Dense Layer . . . . .                                      | 28        |
| 4.4.7    | ReLU (Rectified Linear Unit) Activation Function . . . . . | 28        |
| 4.4.8    | Softmax Activation Function . . . . .                      | 28        |
| 4.4.9    | Parameters Calculation . . . . .                           | 29        |
| 4.5      | Implementation Of Our Machine Learning Model . . . . .     | 31        |
| 4.5.1    | Best performing Ensemble Model . . . . .                   | 32        |
| <b>5</b> | <b>Experimental Results</b>                                | <b>34</b> |
| 5.1      | Overview . . . . .                                         | 34        |
| 5.2      | Hardware Study . . . . .                                   | 34        |
| 5.3      | Performance Metrics . . . . .                              | 35        |
| 5.4      | Deep learning Approach . . . . .                           | 36        |
| 5.4.1    | Text Classification Model with Bi LSTM . . . . .           | 36        |
| 5.5      | Machine Learning Approach . . . . .                        | 40        |
| 5.5.1    | Extra Trees Classifier . . . . .                           | 40        |
| 5.5.2    | Random Forest Classifier . . . . .                         | 41        |
| 5.5.3    | Extreme Gradient Boosting Classifier . . . . .             | 42        |
| 5.5.4    | Bagging Classifier Classifier . . . . .                    | 43        |
| 5.5.5    | Logistic Regression . . . . .                              | 44        |
| 5.5.6    | Decision Tree Classifier . . . . .                         | 45        |
| 5.5.7    | Support Vector Machine . . . . .                           | 46        |
| 5.5.8    | K-Nearest Neighbor Classifier . . . . .                    | 47        |
| 5.5.9    | Multinomial Naive Bayes . . . . .                          | 48        |
| 5.5.10   | Ensemble Model . . . . .                                   | 49        |
| 5.6      | Comparison . . . . .                                       | 51        |
| <b>6</b> | <b>Conclusion</b>                                          | <b>53</b> |
| 6.1      | Limitations . . . . .                                      | 53        |
| 6.2      | Future Work . . . . .                                      | 54        |
| 6.3      | Conclusion . . . . .                                       | 55        |
|          | <b>Bibliography</b>                                        | <b>56</b> |

# List of Figures

|      |                                                            |    |
|------|------------------------------------------------------------|----|
| 3.1  | Logistic Regression Curve. . . . .                         | 14 |
| 3.2  | Ensemble Model . . . . .                                   | 15 |
| 3.3  | Bi-LSTM Networks . . . . .                                 | 18 |
| 4.1  | Sample data of Bengali Spam Emails Dataset . . . . .       | 21 |
| 4.2  | Dataset after annotation . . . . .                         | 22 |
| 4.3  | Architecture Of Our Model . . . . .                        | 25 |
| 4.4  | Workflow of our Machine Learning Model . . . . .           | 31 |
| 4.5  | Ensemble Model (Voting Classifier) . . . . .               | 32 |
| 5.1  | Confusion matrix of Bi-LSTM Based Model . . . . .          | 37 |
| 5.2  | Accuracy &Valid Accuracy Curve . . . . .                   | 38 |
| 5.3  | Loss &Valid Loss Curve . . . . .                           | 39 |
| 5.4  | ROC Curve . . . . .                                        | 40 |
| 5.5  | Confusion matrix of Extra Trees Classifier . . . . .       | 41 |
| 5.6  | Confusion matrix of Random Fores . . . . .                 | 42 |
| 5.7  | Confusion matrix of Extreme Gradient Boosting . . . . .    | 43 |
| 5.8  | Confusion matrix of Bagging Classifier . . . . .           | 44 |
| 5.9  | Confusion matrix of Logistic Regression . . . . .          | 45 |
| 5.10 | Confusion matrix of Decision Tree . . . . .                | 46 |
| 5.11 | Confusion matrix of Support Vector Machine . . . . .       | 47 |
| 5.12 | Confusion matrix of K-Nearest Neighbor . . . . .           | 48 |
| 5.13 | Confusion matrix of Multinomial Naive Bayes . . . . .      | 49 |
| 5.14 | Confusion matrix of Ensemble Voting Classifier . . . . .   | 50 |
| 5.15 | Confusion matrix of Ensemble Stacking Classifier . . . . . | 51 |
| 5.16 | Models Accuracy Comparison . . . . .                       | 52 |

# List of Tables

|      |                                                                    |    |
|------|--------------------------------------------------------------------|----|
| 2.1  | Summary of the joint research using DL and ML techniques . . . . . | 8  |
| 5.1  | BI-LSTM Based Model Performance Metrics . . . . .                  | 37 |
| 5.2  | Extra Trees Classifier Performance Metrics . . . . .               | 41 |
| 5.3  | Random Forest Performance Metrics . . . . .                        | 42 |
| 5.4  | Extreme Gradient Boosting Classifier Performance Metrics . . . . . | 43 |
| 5.5  | Bagging Classifier Performance Metrics . . . . .                   | 44 |
| 5.6  | Logistic Regression Performance Metrics . . . . .                  | 45 |
| 5.7  | Decision Tree Classifier Performance Metrics . . . . .             | 46 |
| 5.8  | Support Vector Machine Classifier Performance Metrics . . . . .    | 47 |
| 5.9  | K-Nearest Neighbor Performance Metrics . . . . .                   | 48 |
| 5.10 | Multinomial Naive Bayes Performance Metrics . . . . .              | 48 |
| 5.11 | Ensemble Model Performance Metrics . . . . .                       | 50 |
| 5.12 | Models Accuracy Comparison . . . . .                               | 51 |



# Abstract

Email, short for electronic mail, facilitates digital communication over the Internet. Billions of emails are exchanged daily globally, heightening susceptibility to threats like spam. Spam emails, often sent by advertisers, scammers, or malicious entities, aim to promote products, services, or fraudulent schemes. Bangla lacks effective spam email detection techniques, posing risks to Bengali speakers. This study aims to categorize Bangla emails using machine learning and deep learning. Data collection and categorization into spam, ham, and professional groups precede algorithm training. Various classifiers including Decision Tree, Random Forest, Logistic Regression, ExtraTrees Classifier, Bagging Classifier, XGBClassifier with ensemble models using soft voting yielding the highest accuracy at 89.34% by using word embedding techniques like TF-IDF, BOW, and Word2Vec are employed. Additionally, a Bi-LSTM-based deep learning model achieves Train accuracy 99.97% and Test Accuracy is 94.46%. All things considered, the proposed system may improve email security and privacy for Bangla-speaking individuals and businesses, reducing the hazards connected to spam emails in the online environment.

**Keywords:** *Bangla Email Classification, Spam Mail, Deep Learning, Long Short Term Memory (LSTM) , Bidirectional LSTM, Machine Learning, Ensemble Model*

# CHAPTER 1

## INTRODUCTION

### 1.1 Introduction

Communication Email facilitates quick communication, allowing people to send messages and information around the world in seconds. This quick sharing of information increases productivity and efficiency in both personal and professional settings. Email still dominates personal communication even in the age of social media and instant messaging. In contrast to other avenues of communication, it offers a certain level of formality and secrecy. It makes it possible for people and organizations to securely exchange private or sensitive information. According to recent data, 361.6 billion emails will be sent and received globally each day by 2024. Anyone with an email account has likely received unsolicited emails in their spam folders at some point. It is estimated that 60 billion spam emails will be sent each day.

Although most individuals find spam bothersome, they accept it as an unavoidable consequence of email communication. Spam can be dangerous in addition to being bothersome since, if improperly filtered and frequently removed, it can choke email inboxes. Spammers, or senders of unsolicited emails, frequently change their approaches and content in an attempt to deceive potential victims into downloading malware, exchanging personal information, or donating money. The majority of spam emails are commercial in nature and have a financial incentive. Spammers attempt to mislead recipients by making fraudulent claims, selling dubious products, and promoting inaccurate information. Because they can simply send their dodgy message to so many email addresses in one stroke, spammers

can con their way to a large payout even though they only expect a small percentage of recipients to engage or interact with their message. Because of this, spam is still a major issue in the contemporary digital economy. Malicious uses for spam emails include phishing for confidential information, disseminating malware, carrying out financial schemes, disseminating false information, and taking part in other dishonest actions. Often, the intention is to take advantage of the recipients for financial gain, identity theft, or computer system compromise in order to carry out more harmful acts. This paper explores the detection of phishing emails with the goal of offering a thorough examination of existing methods, difficulties, and new developments. As cyber thieves consistently modify their tactics to deceive users, detecting and combating phishing threats needs a multidimensional approach that incorporates technology advancements, user education, and proactive strategies. Organizations may strengthen their cybersecurity defenses and protect against the potentially disastrous effects of phishing scams by comprehending the nuances of phishing assaults and keeping up with changing detection technologies. Because of this, the purpose of this paper is to clarify the efficacy of phishing email detection through the application of machine learning and deep learning models. These days, it is feasible to identify spam or phishing emails by using both machine learning models like ExtraTreesClassifier, Random forest, Decision tree, SVM, LogisticRegression, GradientBoosting, multinomial NB, KNN, Ensemble model classifier like voting and stacking and deep learning models like LSTM, BiLSTM. The effectiveness of our models like Random Forest, Support Vector Machine (SVM), Extra Trees Classifier, LogisticRegression, Bi-LSTM based deep learning model is examined in this study.

The research investigates the nuances of each model's methodical approach to spam email identification. The goal of the project is to create reliable spam detection systems that can protect email security in the digital era and improve user experience by experimenting with datasets and optimizing algorithms. Our research indicates that many people are currently receiving a large volume of unsolicited emails and spam in the Bangla language. Currently, accessible email service providers like Yahoo and Gmail dreadfully failed to identify scam emails written in Bangla. With over 234 million speakers globally, Bangla (endonym Bengali) is the seventh most spoken language in the world. Over the past ten years, there has been a significant surge in the number of internet users in Bangladesh particularly among persons who speak Bengali. This encourages us to develop a system for detecting Bangla spam mail.

## 1.2 Motivation

The motivation behind Bangla spam email detection is to enhance user experience, improve security, optimize resources, maintain infrastructure integrity, build trust, comply with regulations, and reduce operational costs for email service providers and users alike. to enhance the user experience as a whole and guarantee that consumers only receive authentic and pertinent communications. In order to protect consumers from possible security risks and to preserve sensitive data, financial information, and personal information, spam email detection is essential. Email systems can manage their resources, ensuring efficient operation and minimizing the influence on system performance, by efficiently recognizing and filtering out spam. Spam detection and prevention contribute to the integrity of email systems, making sure they run smoothly and dependably. Email service companies work hard to earn users' trust. Effective spam identification lowers the possibility that users will come across fraudulent or misleading messages, which contributes to the trustworthiness of email services. Managing spam requires a lot of computing power, storage, and IT support. These administrative expenses related to handling spam-related problems can be decreased with the use of effective spam detection.

## 1.3 Objective

**1. To Detect Bangla spam emails:** In order to provide Bengali-speaking users with a safe and pertinent email experience, it is important to identify spam emails in Bangla and develop a customized, efficient spam filtering system that takes into account the linguistic and cultural characteristics that are specific to the Bengali language.

**2. Filtering Unwanted Content:** To stop unsolicited and irrelevant emails, or "spam," from getting to users' inboxes, identify and filter them out.

**3. Security:** In order to shield users from potential security risks, identify and prevent emails that include dangerous material, such as phishing links, malware, viruses, and scams.

**4. To Compare our proposed model with existing work:** Comparing the performance of the suggested classification model in relation to previous research in the area of Bangla spam email detection is another significant goal of the present research. By executing this comparative analysis, the study seeks to verify the established model's uniqueness and efficacy. In the end, this goal advances the discipline by advancing related research initiatives and the development of spam email detection methods.

## 1.4 Operating System and IDE

The research was conducted on the Windows 10 operating system. The following Integrated Development Environments (IDEs) were utilized:

- Kaggle
- Jupyter Notebook.
- Google Coolab

## 1.5 Application

- **Filtering Unwanted Content:** The main function of the application is to filter out uninvited and possibly harmful emails so that users' inboxes are kept free.
- **Protection Against Phishing:** Phishing emails that try to trick people into sending sensitive information—like passwords, credit card numbers, or personal information are easier to spot and stop when spam filters are used.
- **Preventing Malware Distribution:** Malware is frequently delivered through spam emails. Detection systems can shield users from malware infestations by recognizing and blocking emails that contain harmful attachments or links.
- **Improving Email System Performance:** Detection systems improve the overall effectiveness and speed of email servers by lowering the amount of spam emails. This guarantees a faster processing time for valid emails.

## 1.6 Summary

The detection and prevention of spam emails is the subject of this thesis, which focuses specifically on Bangla spam. The research uses machine learning and deep learning models driven by the desire to improve user experience, strengthen security, and lower operating expenses. It tackles the difficulties in spam identification caused by variations in the Bengali language. Using tools like Kaggle and Jupyter Notebook for Windows 10 experimentation, the research creates a powerful spam filtering system. In order to establish a safer and more effective email ecosystem, feasible uses include blocking undesired information, guarding against phishing, limiting the spread of malware, and enhancing email system accuracy.

## CHAPTER 2

## LITERATURE REVIEW

### 2.1 Overview

This literature review explores various studies on spam detection in Bengali text, covering machine learning and deep learning approaches. Despite variances in datasets and methodology, these research collectively help to progress spam detection in Bengali text, underlining the necessity of robust and flexible systems in facing developing spam methods. We will evaluate five research papers from various sources that align with our approach to Spam email detection.

### 2.2 Literature Review

Detecting and filtering spam emails remain a crucial challenge in modern email systems, that demand robust and adaptive approaches to keep pace with evolving spam tactics. The following section summarizes all prior research on machine learning and deep learning models for spam email detection and classification. A small group of researchers identified spam emails in Bangla by collecting datasets from multiple sources and using machine learning and deep learning models to identify spam emails. For their work, Riyadil Zannat et al. [1] proceeding CNN, Bangla-BERT, and Bi-LSTM models, and the total number of emails was 5572 where 4572 were ham and 1000 were spam emails. In the Data pre-processed part, The key takeaway is that to maximize the amount of data given into the deep learning model, data cleaning methods built primarily for Bangla sentiment analysis are used. For feature extraction, here one hot encoding, word embedding, and count vectorizer

are used. And Bi-LSTM classifier performs better than the other models examined, with an accuracy of 97.04% in combination 1 (epoch 100, learning rate 0.001%). The accuracy of the CNN and Bangla BERT classifiers is comparable, with 96.8% and 96.4%, respectively. This comparison suggests that the Bi-LSTM model is the most suitable model for sentiment analysis in their study. The previous study, which only used machine learning techniques to detect spam emails in Bangla, achieved a maximum accuracy of 93.60%. It was claimed that the Random Forest provided the highest level of accuracy. The deep learning-based methodology used in this study allowed the Bi LSTM to reach the highest accuracy of 97%.

In 2019, Ruhul Amin et al. [2] used their dataset which are collected from various source and also their personal emails. Their dataset contains 4766 train datasets and 850 test datasets. and is divided into two categories. Used supervised ML methods, namely Multinomial Naive Bayes, Decision Tree (DT), K Nearest Neighbor (KNN), Random Forest (RF), AdaBoost, and SVM to train a model. Text pre-processing, feature extraction, model training, and testing were all part of their methodology. Accuracy, Sensitivity (recall), and Specificity were used to assess the performance of each algorithm. K-Fold Cross-Validation (CV) is used to measure performance evaluation. After their accuracy test, the result came out as Random Forest emerged as the best performer with an accuracy of 93.60%, followed by SVM at 90.66%.

Md Mohsin Uddin et al. [3] proceed with their work where the dataset has been prepared by collecting SMS messages from mobile phones. In the prepared Bengali SMS spam dataset, the spam texts comprise of 22% and the ham texts comprise 78%. The corpus is randomly shuffled and divided into two parts. Then, the first 80% of SMS messages are separated for training and the rest 20% of messages are used for testing. Traditional machine learning- Naïve Bayes, Logistic Regression, SVM. Deep learning- RNNs (LSTM, GRU) with different activation functions (ReLU, Sigmoid, TanH) and optimizers (RMSPROP, ADAMAX, ADAGRAD, ADADELTA, SGD). Accuracy-99% for both LSTM and GRU, significantly higher than traditional algorithms (93-96%). F1-score: 0.96 for LSTM-TanH-ADAGRAD, 0.96 for LSTM-ReLU-ADAGRAD, 0.96 for GRU-SIGMOID-ADAGRAD, again outperforming baselines. RNNs, specifically LSTM and GRU, are effective for detecting Bengali SMS spam with high accuracy. ADAGRAD optimizer achieved the best performance among all deep learning models.

Tanvirul Islam et al. [4] used a dataset in this research as a collection of Bangla sentences from social media platforms. The dataset consisted of 1965 instances, including 646 spam and 1319 ham. they collected data and Extracted comments from Facebook groups/pages and Youtube videos using tools like "socialfy" and "comments". Pre-processed the data by

Removing punctuation marks, numerical values, and emoticons. Labeled data as "spam" (1) or "not spam" (0). Extracted feature where Used TF-IDF to analyze the importance of words based on their frequency within documents. Trained a Multinomial Naive Bayes classifier on the pre-processed data. Achieved an accuracy of 82.44% in identifying spam Bangla text content. The confusion matrix showed higher accuracy for predicting "not spam" due to the imbalanced dataset (more non-spam examples). Precision, Recall, F1-score, and AUC metrics confirmed reasonable performance. Overall, their methodology showed promising results in detecting malicious Bangla text content using social media data and machine learning techniques, which is comparable to our current work .

Shovon Ahammed et al. [5] This study, which focuses on creating a new dataset and utilizing machine learning methods for classification, examines the identification of hate speech in the Bangla language. Facebook is a well-known social networking site in Bangladesh. The dataset was gathered from there and classified into categories for hate speech and non-hate speech. Preprocessing was done on the data to address things like negations, emojis, and typos. Support Vector Machine (SVM) and Naïve Bayes were the two machine learning methods used for classification. While Naïve Bayes obtained an accuracy of 72%, the SVM algorithm only managed a 70% accuracy. In terms of identifying hate speech in Bangla, both systems fared well; however, Naïve Bayes showed marginally better accuracy. In general, the study aids in the identification of hate speech in the Bangla Language.

Ovishake sen et al. [6] While being the world's sixth most spoken language, Bangla struggles NLP issues due to limited resources. Recent attempts have evaluated 75 academic publications from several disciplines, including sentiment analysis, machine translation, and speech recognition. These papers address both classical and modern approaches to Bangla NLP, with the aim of moving the discipline ahead and motivating future study.

## 2.3 Summary



Table 2.1. Summary of the joint research using DL and ML techniques

| Authors and Reference      | System Description                                                                               | Dataset                                                                                                                                                                | Models Used                                                             | Detection Rate (Accuracy)                                                                                                                                                        |
|----------------------------|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Riyadil Zannat et al. [1]  | A Deep learning based approach for detecting bangla spam emails                                  | A dataset of size 5572 was created Through posting to several social media research groups and using publicly available databases, emails were gathered for this study | CNN, Bangla-BERT and Bi-LSTM                                            | In combination 1, Bi-LSTM classifier outperforms other models with 97.04% accuracy at epoch 100 and learning rate 0.001%.                                                        |
| Ruhul Amin et al. [2]      | A bangla spam email detection and datasets creation approach based on machine learning algorithm | The training dataset comprises a total of 4766 Bangla emails and The test dataset consists of 850 Bangla emails                                                        | Multinomial Naive Bayes , KNN,Decision Tree,SVM, Random Forest,AdaBoost | (MNB): 88.69% (KNN): Not provided Decision Tree: 89.18% (SVM): 90.66% RFt: 93.60% Ad-aBoost: 89.51%                                                                              |
| Md Mohsin Uddin et al. [3] | Detecting bengali spam sms using Recurrent Neural Network                                        | Bengali SMS dataset from mobile phones with 22% spam & 78% ham messages. The dataset was shuffled and split, with 80% for training and 20% for testing.                | Naïve Bayes, SVM, LR, LSTM, GRU, RNN                                    | Naïve Bayes: 93% SVM: 95% LR: 96% Basic RNN: Accuracy: 97% F1-Score: 0.94 LSTM: Accuracy: 99% F1-Score: 0.96 GRU: Accuracy: 99% F1-Score: 0.96                                   |
| Tanvirul Islam et al. [4]  | using Social Networks to Detect Malicious Bangla Text Content                                    | The dataset totaled 1,965 instances, with 32.87% spam (646 instances) and 67.13% ham (1,319 instances).                                                                | Multinomial Naive Bayes                                                 | The accuracy of the Multinomial Naïve Bayes classifier for detecting spam Bangla text content was reported to be 82.44%.                                                         |
| Shovon Ahammed et al. [5]  | Implementation of Machine Learning to Detect Hate Speech in Bangla Language                      | The paper collected over 5,000 data points but balanced their dataset to 1,339 samples, including 665 hate speeches and 674 neutral speeches.                          | Support Vector Machine (SVM) and Naïve Bayes                            | SVM: Accuracy 70%, Precision 0.68, Recall 0.70, F1-score 0.69 for hate speech class. Naïve Bayes:Accuracy 72%, Precision 0.70, Recall 0.74, F1-score 0.72 for hate speech class. |

## CHAPTER 3

## BACKGROUND STUDY

### 3.1 Overview

This research aims to build a system to find Bangla spam emails by studying the Bangla language itself, seeing how other languages handle spam, and figuring out what's especially difficult (challenges) or helpful (opportunities) for dealing with Bangla spam. In this research we are going to explain what email spam is and how it affects people and businesses and the significance of methods for detecting spam. Then, we justify the use of deep learning and machine learning in spam detection. The issue is the rise in spam emails that are directed towards Bengali users. These emails might be spam, phishing attempts, or promote unwanted products. Spammers get around traditional filters by taking advantage of the language barrier. The existing solution is Conventional spam filters might not translate well to Bangla since they frequently rely on keywords or sender reputation. Studies on the identification of spam in other languages, especially those that have features (such as complicated script and agglutinative morphology) with Bangla.

### 3.2 Type of Mails

We categorized our data into three groups: spam, ham, and professional. Our work contributes to the creation of a new dataset for the detection of spam emails in the Bangla language, which is then detected using machine learning and deep learning algorithms.

### 3.2.1 Spam Emails

We have selected some types of spam emails like:

- **Money Scams::** In Money Scams are some spam emails which target credit card information, mobile banking information. Fraudulent emails sent by scammers with the intention of deceiving recipients into providing money, personal information, or financial details under false pretenses. Large sums of money, profitable business possibilities, or financial assistance are frequently offered in these scams, but their true purpose is to defraud victims of their money or sensitive information.
- **Fake Lottery:** Fake Lottery is one of the most famous and old spam email types. The spammer entices the receiver by saying that he has won the lottery.
- **Harmful Ads:** In Harmful Ads are ads of those types of products that contain harmful chemicals.
- **Adult content:** Adult content are website links of porn sites. When the user clicks on those links it takes them to porn sites or some harmful websites that forces users to install unwanted harmful applications.

### 3.2.2 Ham Emails

Then, we have selected some types of ham emails like:

- **Educational Mails:** It is the most useful for learning. Teachers, professors, online learning platforms, and even educational institutions can utilize them to assign assignments, communicate with students, and provide information in an effective and simple method.
- **Official Mails:** The Official mails are sent and received for business purposes by coworkers, managers, or outside contacts. They are frequently used to send either virtual or in-person meeting invitations. These emails can be used to communicate with customers in a variety of ways, such as answering questions, resolving issues, giving order updates, or making promotional offers.
- **Retail Mails:** In Retail mail refers to the various types of email communication that retail businesses use to engage with their customers, promote products, drive sales, and enhance customer relationships.

### 3.2.3 Promotional Emails

Now, we have selected some types of promotional emails like:

- **Offers for Sales and Discounts:** The recipients of these emails are granted access to sales events, special discounts, and promotions. In order to attract attention recipients to take advantage of the offer, they frequently feature alluring subject lines and images.
- **New Product Launches:** Businesses use promotional emails to introduce new products or services to their audience. These emails highlight the features and benefits of the new offering and may include special introductory pricing or incentives to encourage purchases.
- **Flash Sales offers:** These emails advertise limited-time or flash sales, which are usually available for a short time (e.g., 24 or 48 hours). Urgency is emphasized to encourage immediate action from recipients.
- **Giveaway:** These emails offer recipients free products, samples, or gifts with purchase. Giveaways and freebies can build buzz around a brand or product launch, reward repeat business, and draw in new clients.

### 3.3 Significance of the work

There is much research work on spam text of other languages. But there is no significant work on Bangla text. Bangla language is one of the most popular languages in the world. Bangla is the eighth most spoken language in the world, according to a Dhaka Tribune report [Dhaka tribune]. The internet users of Bangla language are many. They can communicate with each other by sending bangla email. So, all of these facts encouraged us to work with Bangla spam text.

### 3.4 Challenges of Bangla Spam Detection

Dataset played a crucial part in our work. As our work is on Bangla spam email. We faced a problem because there were no publicly available Bangla spam email datasets for training models. Forming the dataset was a challenging part of our work. We have used email spam folders from mobile, Ai generated, survey to collect the spam data. Then, we used online documents like blog, personal email, Ai generated, survey to collect the ham data. We have also used an email promotional folder, meta ad library, survey to collect the promotional data. After collecting the data, the next challenge was to train the data to detect Bangla spam mail. Having trouble identifying spam features unique to Bangla like:

- Usage of emojis, slang, and casual language.
- Cultural allusions and Bangladesh-specific marketing strategies.
- Spammers utilize obfuscation techniques or encoded characters.

It was difficult to get the best outcome from the algorithms. We have attempted to extract the best possible result from the dataset by fine-tuning several algorithmic aspects.

## **3.5 Bangla spam email detection algorithms that we used**

### **3.5.1 Machine Learning**

Several classification algorithms have been used in the experiments namely KNN, Decision Tree, SVC, MNB Random Forest, AdaBoost, LogisticRegression, BaggingClassifier, ExtraTreesClassifier, XGBClassifier and ensemble model classifier like Voting and stacking classify Bangla emails as spam , ham or promotional. And in Word embedding process we use TF-IDF, BOW and Word2Vec.

#### **3.5.1.1 Decision Tree**

This algorithm is easy to understand and uses a series of queries regarding the properties of the data to determine if it is spam, ham or promotional.[7] Decision trees have the ability to learn decision rules for Bangla email spam detection based on features that are derived from Bangla text, such as word frequency, the presence of specific keywords, or the email's structural elements. We scaled the data to help the model learn better. Scaling data is a beneficial technique since it makes it easier to learn the model. It speeds up training and improves the mistake surface form.

#### **3.5.1.2 Random Forest**

The Random Forest algorithm as a meta-learner, with many individual trees. Each tree contributes to the overall classification by voting on the dataset, and the algorithm chooses the classification with the most votes [8]. Each decision tree is constructed using a random subset of the training data and a procedure known as replacement, in which certain entities are sampled several times while others do not appear at all. Furthermore, at each node, a model based on a different random subset of the training data and variables is used to find the best dataset partitioning. Importantly, no trimming is done, and each decision tree is grown to its full size. Finally, the Random Forest produces an ensemble model made up of several decision tree models, each of which contributes a vote to the final conclusion, with the majority choice taking precedence.

### 3.5.1.3 Support Vector Machine

Support Vector Machine is a supervised machine learning algorithm which can be used for both classification or regression challenges [9]. However, it is mostly used in classification problems. In the SVM algorithm, we plot each data item as a point in n-dimensional space (where n is a number of features you have) with the value of each feature being the value of a particular coordinate.

### 3.5.1.4 Multinomial Naive Bayes

Multinomial Naive Bayes (MNB) is a widely used and effective machine learning technique based on Bayes' theorem. It is a probabilistic classifier that calculates the probability distribution of text input, making it ideal for data with features representing discrete frequencies or counts of events in a variety of NLP applications [10]. Thomas Bayes invented the Bayes theorem, which calculates the likelihood of an event based on prior knowledge of its circumstances. When predictor B is provided, we determine the chance of class A. It is based on the following formula:  $P(A|B) = P(A) * P(B|A)/P(B)$ . It is straightforward to implement since all you need to do is compute the probability. This method works for both continuous and discrete data. And multinomial NB is computationally efficient and Easy Implementation.

### 3.5.1.5 Logistic Regression

By estimating probabilities using a logistic function, logistic regression analyzes the connection between the categorical dependent variable and one or more independent factors [11]. From the definition it seems, the logistic function plays an important role in classification here but we need to understand what is logistic function and how does it help in estimating the probability of being in a class.

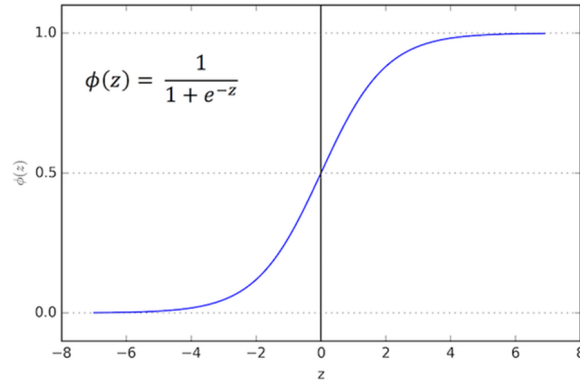


Figure 3.1. Logistic Regression Curve.

The graph and formula shown in the image above are referred to as the Sigmoid curve and the logistic function, respectively. An S-shaped curve is produced by the sigmoid function. The sigmoid function's output goes to zero as  $z$  approaches  $-\infty$  and to one as  $z$  approaches  $\infty$ .

#### 3.5.1.6 k-nearest neighbors (KNN) Ensemble Model

KNN is a supervised learning technique for classification and regression that categorizes input based on its similarity to neighbours [12]. It keeps all training data and sorts fresh data points into the most comparable group. It uses proximity to calculate distances between places and selects the k-nearest neighbours. Majority voting is used for classification, whereas neighbour labels are aggregated for regression. It is nonparametric and makes no assumptions about data distribution. KNN is referred to as a lazy learner since it does not immediately learn from the training set before acting on it during classification. K-NN can respond to changes in the dataset without the requirement for retraining. This makes it suited for dynamic contexts where data distributions might change over time. Handling Mixed Data Types making it suitable for a wide range of datasets. K-NN is simple to implement and understand, making it suitable for beginners in machine learning. The variable K value, which represents the number of neighbours, can have a substantial impact on the classifier's performance. Experimenting with various K values enables us for fine-tuning the model to obtain the best results for a particular dataset.

### 3.5.1.7 Ensemble Model

Ensemble Modeling is a method for combining multiple machine learning models to enhance overall prediction performance [13]. The core premise is that a group of weak learners might combine to generate a stronger learner. An ensemble model is generally composed of two steps: Several machine learning models are trained individually. Their forecasts are gathered in some way, such as by voting, averages, or weighting. The ensemble is then utilized to create the final forecast. It works by integrating the forecasts of several distinct models to get a final prediction. There are several methods for integrating these forecasts, including averaging them or allowing each model to "vote" on the final result. Ensemble approaches, which take use of the different views of individual models, frequently result in greater performance and more solid predictions than any one model. It's similar to combining the expertise to arrive at a better decision.

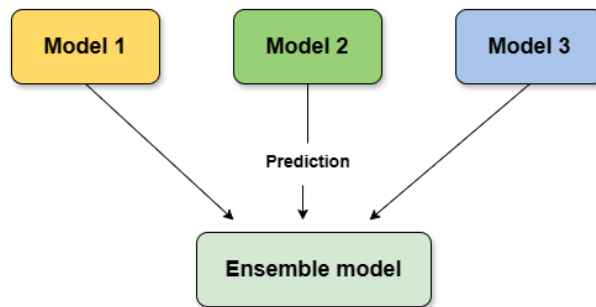


Figure 3.2. Ensemble Model

There are several strategies for combining predictions:

- **Voting:** Each base classifier's prediction is treated as a "vote", and the final prediction is determined by majority voting.
- **Stacking:** Training a meta-classifier on the predictions of the base classifiers to make the final decision.
- **Bagging (Bootstrap Aggregating):** Training multiple models on different subsets of the data and averaging their predictions.
- **Boosting:** Sequentially training models, with each subsequent model focusing on examples that the previous ones struggled with.



#### 3.5.1.8 AdaBoost

AdaBoost in machine learning is one of these predictive modeling algorithms. AdaBoost, also known as Adaptive Boosting, is a machine learning technique used as an Ensemble Method [14]. It iteratively trains a series of weak classifiers using different subsets of the training data. During each iteration, the algorithm gives higher weights to the misclassified samples from the previous iteration, focusing on the more difficult cases. This procedure enables the weak classifiers to pay more attention to previously misclassified examples, improving their performance. The AdaBoost Work operating procedure is

- Step 01: Initialize the sample weights for each training example For every iteration.
- Step 02: Train the weak classifier using the current sample weights.
- Step 03: Calculate the error of the weak classifier
- Step 04: Determine the weight of the weak classifier depending on the mistake.
- Step 05: Update the sample weights depending on the poor classifier's performance.
- Step 06: Normalize the sample weights.
- Step 07: End the iterations.
- Step 08: Combine the weak classifiers using a weighted majority vote.

#### 3.5.1.9 BaggingClassifier

Bagging classifier is an ensemble strategy in which many models are trained independently on random subsets of data before aggregating their predictions by voting or average [15]. Bagging (bootstrap aggregating) is a strategy for increasing the accuracy of predictions generated by a supervised learning system. The main concept is to train a variety of models on different randomly selected subsets of the training data, and then aggregate their predictions using a voting mechanism. The main advantages of bagging is Reducce overfitting and Decreases model cariance. It is Handles high variability and The averaging process helps in reducing the noise in the final prediction. The main advantage is Handles imbalanced data. Now, The Bagging classifier operating procedure is

- Step 01: Start with the original dataset.
- Step 02: Begin iterations process.
- Step 03: In each iteration, sample N instances with replacement from the original training set.
- Step 04: Apply the learning algorithm to the sampled dataset to train the base classifier

- Step 05: Store the classifier from the current iteration.
- Step 06: Stop the process when the desired number of base estimators (or iterations) is reached.
- Step 09: Combine the predictions from all the sampled models through simple averaging (for regression) or voting (for classification) to make the overall prediction.
- Step 10: The final prediction is made by aggregating the predictions of each base estimator.

#### 3.5.1.10 ExtraTreesClassifier

Extra Trees Classifier) is an ensemble learning approach that uses the outputs of several de-correlated decision trees gathered in a "forest" to provide a classification result [16] . Extra Trees is similar to Random Forest in that it generates several trees and splits nodes using random subsets of features, but there are two significant differences: it does not bootstrap observations (meaning it samples without replacement), and nodes are divided using random splits rather than optimal splits. In conclusion, ExtraTrees: builds many trees with bootstrap = False by default, which indicates that it samples without replacement. Nodes are split based on random splits among a random subset of the attributes chosen at each node. Randomness in Extra Trees is generated via random splits of all observations rather than by data bootstrapping.

#### 3.5.1.11 XGBClassifier

XGBoost (eXtreme Gradient Boosting) is a more sophisticated implementation of the gradient boosting method that has garnered popularity for its superior performance and efficiency in a variety of machine learning tasks [17]. We mostly utilize XGBoost since it has numerous key properties that make it perfect for classification jobs. XGBoost is tuned for speed and efficiency, making it suitable for huge datasets and real-time applications. XGBoost includes L1 (Lasso) and L2 (Ridge) regularization terms to reduce overfitting and improve generalization. Furthermore, XGBoost can handle missing data automatically, which reduces the requirement for preprocessing and imputation. It enables the user to do cross-validation for each iteration of the boosting process.

### 3.5.2 Deep Learning

#### 3.5.2.1 Long Short-Term Memory(LSTM)

The purpose of Long Short Term Memory (LSTM) is to address the deficiencies of standard RNNs by enabling the network to store information in a memory module, facilitating access to past data when needed.[18] When we use Lstm for natural language processing (NLP) tasks. we have several advantages like Handling sequential data, Handling long-term dependencies, a large amount of data, variable-length inputs. And When LSTM networks are combined with an attention mechanism, they can focus on specific parts of the input sequence. In the LSTM here are some drawbacks like Computational complexity, Overfitting, Limited context, Difficult to interpret and Long-term dependencies. The implementation of LSTM for text classification in NLP begins with translating the text into numerical representations using tokenization and word embedding techniques. Tokenization divides the text into individual words, whereas word embedding allocates high-dimensional vectors to words, effectively maintaining their semantic meanings.

#### 3.5.2.2 Bidirectional LSTM

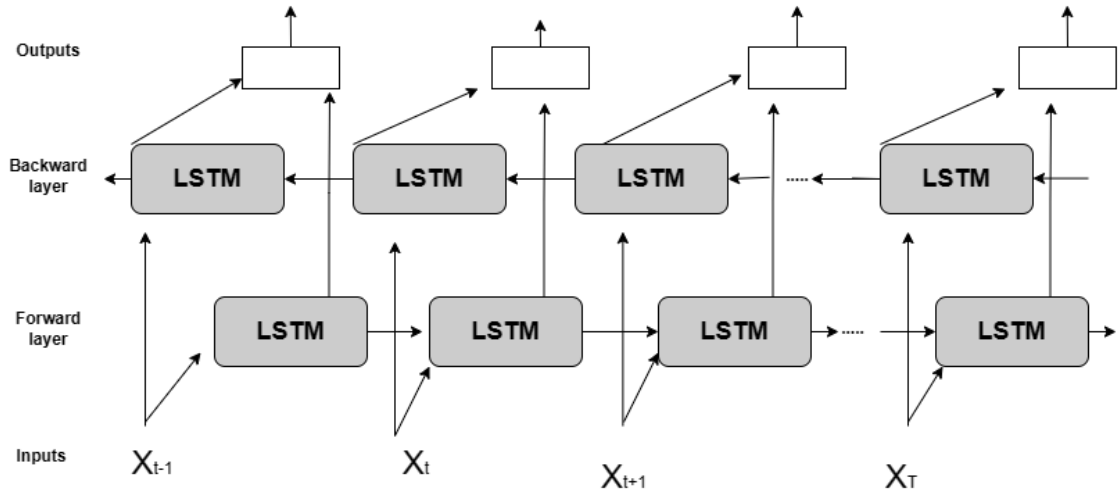


Figure 3.3. Bi-LSTM Networks

Recurrent neural networks of the bidirectional LSTM (BiLSTM) kind are mostly employed in natural language processing. BiLSTM uses data from both ends to process input in both directions, in contrast to standard LSTM. Because of this[19] it is effective at recording the word-to-sentence linkages across the whole sequence. By adding an additional LSTM layer,

BiLSTM flips the information flow. This basically indicates that the extra layer processes the input sequence in reverse. Subsequently, techniques such as averaging, summing, multiplying, or concatenating are applied to integrate the outputs of both LSTM layers. To put it simply, BiLSTM is adding another LSTM layer and flipping the information flow. As a result, the additional LSTM layer processes the input sequence in reverse. After that, the outputs from the two LSTM layers are combined using a number of methods, including concatenation, multiplication, averaging, and summing.

#### **3.5.2.3 Analysis**

Deep Learning achieves higher accuracy than ML with larger datasets and captures complex relationships within the data. However, training can be time-consuming and require significant computational resources.

#### **3.5.2.4 Benefits of Bangla Email Spam Detectio**

- We can reduce time spent by removing unnecessary emails.
- We are able to protect users from malware and phishing frauds.
- We can increase the security and trust of Bangla internet communication.
- We are able to keep our inboxes organized and in control.

#### **3.5.2.5 Additional considerations for Bangla email spam detection**

Spammers are always devising new strategies to get around spam filters. This implies that any Bangla spam detection system must have the flexibility to adjust and pick up fresh spam email examples. One of the biggest challenges in spam filtering is avoiding false positives. These are emails that are mistakenly identified as spam. False positives can be a real problem for users, as they may miss important messages. When designing a Bangla spam detection system, it's crucial to find a balance between catching spam and avoiding false positives. This might involve using techniques like allowing users to report false positives so the system can learn from them.

By conducting a thorough background study encompassing these aspects, we can use machine learning and deep learning techniques to create a reliable and efficient system for detecting spam emails in Bangla.

## CHAPTER 4

## METHODOLOGY

### 4.1 Introduction

Email has emerged as one of the main means of communication in the modern digital era, for both personal and business needs. However, with the widespread use of email, there has been a corresponding increase in unwanted or unsolicited emails, commonly known as spam. Spam emails can be harmful in addition to cluttering inboxes because they may include viruses or phishing attempts. One of the languages spoken most widely in the world is Bangla, or Bengali, which is mostly spoken in Bangladesh and the Indian state of West Bengal. There is more Bangla-language content online, including spam emails, as a result of these areas' growing internet penetration. Detecting spam in Bangla presents unique challenges due to linguistic nuances and variations specific to the language.

This project aims to detect spam email in Bangla language. This paper presents a methodology for Bangla spam email detection, focusing on Data collection, preprocessing, feature engineering, machine learning techniques, deep learning techniques and text classification algorithms. Applying machine learning and deep learning techniques, we aim to develop effective system capable of accurately identifying and filtering out spam emails written in Bangla. This research significantly advances the field of Bangla spam email detection by showing the practical implementation of cutting-edge ML and DL techniques. The project's outcomes have the potential to revolutionize automated Bangla spam filtering, significantly improving security and user experience for Bangla email users. These advancements can pave the way for more effective detection methods in other languages as

well.

Now, We go into our methodology, findings, and analysis and show how this research categorizes Bangla emails as spam, ham, or promotional.

## 4.2 Dataset Collection Procedure

Our strategy is to create a dataset of Bangla spam email. Then using that dataset to detect spam email in Bangla language. So, data collection is the key part of our work. There are available dataset on other languages for spam email. But there is no dataset for Bangla spam emails. So, we decided to make a dataset of our own. We wanted to make a dataset which consisted of all types of spam message, ham message and promotional message. So, we analyzed different platforms to collect them. We have used email spam folders from mobile, Ai generated, survey to collect the spam data. Then, we used online documents like blog, personal email, Ai generated, survey to collect the ham data. We have also used an email promotional folder, meta ad library, survey to collect the promotional data.

We formed the dataset to complete our task. As we have worked with multiclass classification. So, our dataset was divided into three classes: Spam Bangla email, Ham Bangla email, Professional Bangla email. In our dataset we have considered 4 types of spam messages. They are adult content, fake lottery, ads of harmful products, money scams. We have considered 3 types of ham messages. They are Educational mail, Official mail, Retail mail. Then, we considered 4 types of promotional messages. They are Offers for sales and discounts, New product launches, Flash sales offers, Giveaways. The sample data in the Bengali email dataset is illustrated in the below

|   | Email No. | Bangla Mails                                       | Action      |
|---|-----------|----------------------------------------------------|-------------|
| 0 | 1         | প্রিয় রিজু, আমি তোমার গতকালের রিপোর্ট দেখেছি...   | ham         |
| 1 | 2         | খণ্ডকালীন জ্বাবেতন 1000 BDT/দিনাঘরে বসে কাজ :wa... | spam        |
| 2 | 3         | AWS স্টার্টআপ ডে'তে আপনি আসছেন তো? আপনার স্ট...    | promotional |

Figure 4.1. Sample data of Bengali Spam Emails Dataset

In our dataset, we collected 2992 spam emails, 3084 ham emails and 3396 promotional emails. So, the spam texts comprise 31.59 percent and the ham texts comprise 32.56 percent and the promotional texts comprise 35.85 percent. Then, the first 80 percent of email messages are separated for training and the rest 20 percent of emails are used for testing.

To implement ML and deep learning algorithms to classify Bangla emails as spam or ham or promotional, the dataset needs to be cleaned and pre-processed.

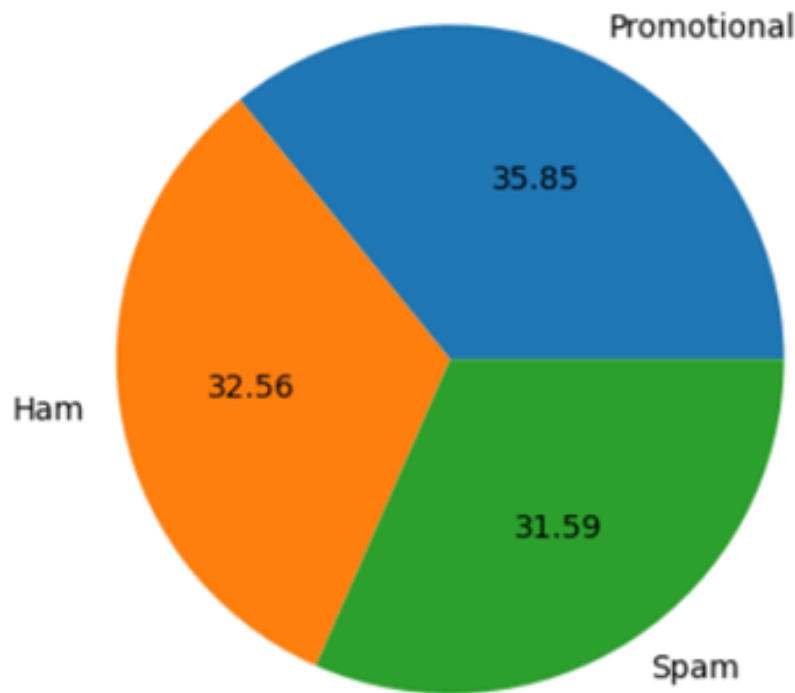


Figure 4.2. Dataset after annotation

## 4.3 Proposed Methodology

### 4.3.1 Data Preprocessing

Data preprocessing is an important step in preparing raw textual data for effective machine learning and deep learning tasks. As we have collected the data, there are different types of anomalies in the data. Using preprocessing we removed the anomalies. In this section, we explain the steps taken to clean, standardize, and enhance the collected sports news dataset for accurate classification.

- **Remove Duplicate Data:** A row that exactly matches another row in the data frame in terms of values is called a duplicate row. To determine how many rows in our data frame were duplicates, we used a method by using `drop_duplicates(keep='first')`.
- **Identify Missing Values:** Identifying missing values in Bangla spam email detection preprocessing involves scanning the dataset for null or empty entries, ensuring completeness and accuracy. Missing values may disrupt analysis and model training,

requiring careful handling through imputation or exclusion to maintain data integrity and algorithm performance. The `.isnull()` method was used to identify which entries had null or missing data.

- **Remove emojis and non-Bangla characters:** It ensures the integrity of the text data by eliminating irrelevant symbols or characters that don't belong to the Bangla script. This stage improves the effectiveness of algorithms in correctly identifying Bangla language patterns typical of spam emails and helps preserve linguistic consistency.
- **Remove Bengali stopwords:** It involves filtering out common words with little semantic meaning, such as articles, prepositions, and conjunctions like (" - "to be"), (" - "that"), (" - "what"), which are unlikely to contribute to distinguishing spam from legitimate emails. This step helps reduce noise in the text data and enhances the performance of machine learning algorithms by focusing on more informative features relevant to spam detection in the Bengali language context
- **Text Data Preprocessing with Regular Expressions:** Regular expressions (regex), an important tool for identifying and manipulating patterns in text data, are used in data preprocessing. They consist of a sequence of characters that when combined create a search pattern that makes it easy for researchers to find and replace specific text features, such as special characters or punctuation. Regular expressions were used in our work to clean and standardize textual data, ensuring consistency and simplifying subsequent data processing activities.

### 4.3.2 Data Annotation

It was the key part of our work. We were focused on annotating our data correctly. As our work finds out if a message is spam or not. So, we made three categories. One category is Ham message, one is Spam message and another is promotional message. Then we labeled the data with tags. If the message had Spam content, we labeled that as Spam and if the comment is appropriate, we labeled that as Ham and if the comment is offers or discounts, we labeled that as promotion. We worked in groups to annotate the data.

### 4.3.3 Feature extraction

#### 4.3.3.1 One-hot encoding

We used one-hot encoding. It represents each class as a binary vector where each class is assigned a unique position in the vector. We labeled data as Ham: [1, 0, 0] Promotional: [0, 1, 0] Spam: [0, 0, 1] In this encoding scheme, each email is labeled with one of these



classes, and its corresponding one-hot encoded vector indicates the presence (1) or absence (0) of each class. This approach facilitates the training of machine learning models by converting categorical labels into a numerical format, enabling accurate classification of Bangla emails into spam, non-spam, or promotional categories.

#### **4.3.3.2 Word Embedding**

We have extracted the features using `CountVectorizer`, `TfidfVectorizer`, and `Word2Vec` classes, which are used for converting text data into numerical feature vectors. `CountVectorizer` converts a collection of text documents into a matrix of token counts, where each row represents a document and each column represents a word in the vocabulary. `TfidfVectorizer` converts a collection of raw documents into a matrix of TF-IDF features, where each row represents a document and each column represents a word weighted by its TF-IDF score. We used the parameter `max_features` as 3000, which specifies that only the top 3000 most frequent words will be considered as features. We converted the Bangla text data into TF-IDF feature vectors using the initialized `TfidfVectorizer`, transforming it into a dense NumPy array from its sparse matrix representation with `toarray`, resulting in the feature vectors stored in the variable `X`. Next we converted the Bangla text data into `CountVectorizer()` using the initialized `Co` matrix. transforming it into a dense NumPy array from its sparse matrix representation with `toarray`, resulting in the feature vectors stored in the variable `X2`. `Word2Vec` preprocessed Bangla mail texts, producing vector representations for each word. These representations are averaged to create a vector for each text. The resulting vectors are stored in a DataFrame column called `Word2Vec`. Finally, a features matrix `X3` is constructed by vertically stacking these `Word2Vec` vectors, enabling further analysis or machine learning applications. We extracted the target labels from the DataFrame `df` encoded, where the labels are contained in columns `'Action ham'`, `'Action spam'`, and `'Action promotional'`, representing non-spam, spam, and promotional emails respectively, storing them in the variable `y`.

#### **4.3.4 Data Splitting**

Effective model training and evaluation require a well-organized dataset division. In this part, we explained how to separate the bangla spam email dataset into discrete groups for training, testing, and validation. We used `scikit-learn`'s `train ,test split` function to partition the dataset `X` (features) and `y` (action labels) into training and testing sets, with a test size of 20 percent, and a specified random seed of 40 for reproducibility.

## 4.4 Working Procedure of Deep Learning

We now discuss the deep learning-based algorithm for identifying Bangla spam mail. To implement this model, we experimented with three different deep learning architectures. Firstly, we created a model using the Bidirectional LSTM. Secondly, we tried add dropout and dense layer. Finally, we implemented a model combining the two Bi-LSTM layer also add dense, dropout and flatten layer. Based on the experimental results that we will explain in the next section, the Bi-LSTM based model gave us the best results in terms of performance. For this reason, here we will limit ourselves to the description of the Bi-LSTM model.

### 4.4.1 Architecture Of Our Model

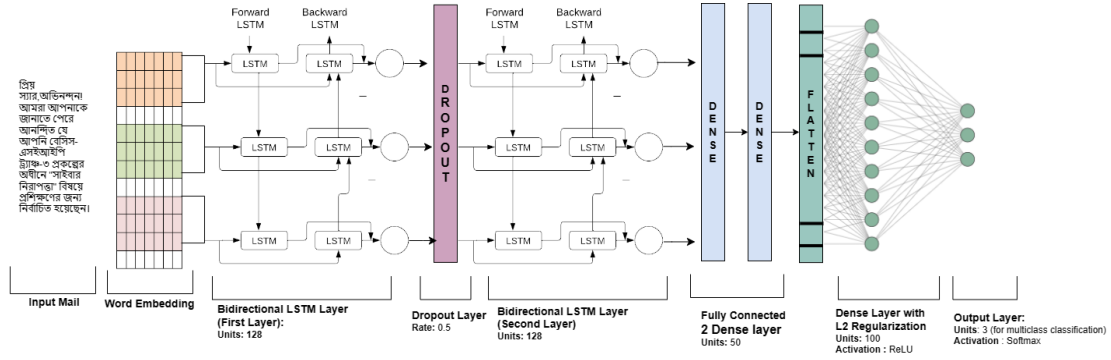


Figure 4.3. Architecture Of Our Model

we describe the architecture and operating principle of the Bi-LSTM based model. The output of the Embedding layer, as described above, is connected with a Bi-LSTM layer. Subsequently, a Bidirectional Lstm layer and Flatten is applied to reduce the dimensionality of the output. Next, we connect another Bi-LSTM layer. Finally, we apply a Dense output layer with a multiple neuron and a Softmax activation function to make decisions for the tree classes spam ham and promotional. In the following, we describe, in detail, the role and configuration of each layer.

### 4.4.2 Embedding layer

The Embedding layer in neural networks for natural language processing converts category text data into dense vector representations, capturing semantic links between words while lowering dimensionality. It learns contextual information and semantic similarity, which improves the model's understanding of text. Using learnable parameters, it modifies during training to optimize word representations for the task. It also supports the use of

pre-trained embeddings for transfer learning, which improves model performance, and provides configuration flexibility to handle diverse vocabularies and jobs.

### 4.4.3 Bidirectional LSTM

The Bidirectional LSTM (Long Short-Term Memory) layer, which is a critical component in sequence modeling tasks such as text classification, expands the capabilities of regular LSTMs by processing input sequences in both forward and backward directions at once. Here's a full explanation of its purpose in our project.[19] Bidirectional LSTM accurately captures the temporal context of input sequences by combining both forward and backward information. This enables the model to grasp the meaning and context of each word by drawing on both preceding and following words. Bidirectional LSTM layer with 128 units to the neural network model. This layer processes input sequences and returns sequences of outputs for each time step. It's commonly used for tasks like sequence-to-sequence prediction, natural language processing, and time series forecasting.

LSTM units in the layer maintain a memory cell capable of retaining information over lengthy sequences. This enables the model to learn relationships between distant words, critical for comprehending the content and context of sentences in text categorization tasks. Bidirectional LSTM is a sophisticated feature extractor that automatically learns hierarchical representations of input sequences. It can detect sophisticated patterns and correlations in text data, allowing the model to differentiate between different types of Bangla emails. The gating methods in LSTMs help to alleviate the vanishing gradient problem, allowing for more stable and efficient training. This enables the model to efficiently propagate gradients across time, resulting in improved learning of complicated patterns and relationships in the data.

The Enhanced Performance Bidirectional LSTM layers, the model gains the ability to capture bidirectional context, which often leads to enhanced performance in sequence modeling tasks. It enables the model to make more informed predictions based on comprehensive contextual information. Bidirectional LSTM layers are flexible and can be stacked or combined with other recurrent layers to create deeper and more sophisticated architectures. This flexibility allows for experimentation with different model configurations to achieve optimal performance. In our Bangla spam mail detection project code, Bidirectional LSTM layers are complemented followed by Dense layers. This integration allows the model to use both local and global information gathered from the input sequences to achieve correct classification. So, bidirectional LSTM layers play an important role in sequence modeling tasks by collecting bidirectional context, learning long-term relationships, and acting as powerful feature extractors, all of which contribute to the

model's capacity to reliably categorize Bangla emails into separate categories.

#### **4.4.4 Dropout layer**

The Dropout layer is employed to prevent overfitting and improve the generalization capability of the model. It is a regularization technique used in neural networks to prevent overfitting. The Dropout layer randomly sets a fraction of input units to 0 at each update during training time. This effectively "drops out" (deactivates) some neurons, reducing the model's reliance on specific neurons and features. The dropout rate is specified by the parameter 0.5, which indicates the proportion of input units that will drop during training. With a 0.5 dropout rate, half of the input units will be set to 0 at random during each training cycle. By randomly removing neurons during training, the Dropout layer helps the network to acquire more robust and generalizable properties. It stops the network from remembering noise or unimportant features in training data, resulting in improved performance on unseen data.

The Dropout layer is usually inserted after a layer with a high number of parameters, such as the Dense or LSTM layers. In our project code, the Dropout layer is inserted after the initial Bidirectional LSTM layer, implying that it seeks to regularize this layer's output to prevent overfitting and improve the model's generalization capabilities. So, the `model.add(Dropout(0.5))` line adds a Dropout layer with a dropout rate of 0.5, which helps to regularize the neural network model and prevent overfitting during training.

#### **4.4.5 Flatten layer**

The Flatten layer in the code snippet is critical in transforming the output of previous multidimensional layers into a one-dimensional representation. It is deliberately placed between these layers and the following fully linked layers to guarantee compatibility for future processing. Flatten simplifies multidimensional data by condensing it into a single vector while maintaining important information. This transformation allows for more efficient feature extraction and abstraction, improving the model's capacity to detect patterns and generate accurate predictions. Unlike trainable layers, Flatten functions only as a reshaping technique, with no learnable parameters. Flatten is a widely used image classification technique that integrates convolutional feature extraction with subsequent classification layers. Its existence facilitates information flow inside the neural network design, greatly adding to the model's

### 4.4.6 Dense Layer

Dense layers are added to the neural network model. We know that Dense layers is known as fully connected layers, connect every neuron in the current layer to every neuron in the subsequent layer. The Activation Function function we used is 'relu' activation function (Rectified Linear Unit) is used for the first three Dense layers ('relu' stands for rectified linear unit). It introduces non-linearity to the network, allowing it to learn complex patterns in the data. In The last Dense layer or output layer with 3 neurons and 'softmax' activation function is the output layer. Softmax activation converts raw scores into probabilities, making it suitable for multi-class classification problems like the one in the provided code where three classes are present ('ham', 'promotional', and 'spam'). The third Dense layer includes a regularization term using L2 regularization with a regularization strength of 0.01. This helps prevent overfitting by penalizing large weights, promoting a simpler model. Overall, Dense layers in the provided code contribute to feature learning and non-linearity introduction in the neural network architecture, culminating in a softmax output layer suitable for multi-class classification, with regularization incorporated to improve generalization

### 4.4.7 ReLU (Rectified Linear Unit) Activation Function

Currently, the ReLU is the most widely utilized activation function in the world. Since it is utilized in nearly all convolutional neural networks for deep learning. [\[20\]](#)

The activation function of the ReLU formula is:  $R(z) = \max(0, z)$ .

The range of zero to infinity

The ReLU Activation Function introduces non-linearity, enabling the network to learn complex patterns and correlations in data. It aids in avoiding the vanishing gradient problem, which can impede the learning process in deep neural networks. It is more computationally efficient than other activation functions, such as sigmoid and tanh. It provides sparse activation in comparison to other activation functions, which can increase network efficiency.

### 4.4.8 Softmax Activation Function

In our bangla spam mail detection project . here we are three type of catagory like ham,spam and promotional [\[21\]](#). So, our project is multiclass classification thats why in the last layer we use activation function is softmax. we know Softmax extends this idea

into a multi-class world. That is, Softmax assigns decimal probabilities to each class in a multi-class problem. Those decimal probabilities must add up to 1.0. This additional constraint helps training converge more quickly than it otherwise would. The formula of Softmax function is -

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

Softmax function normalizes the output scores of the neurons in the output layer, ensuring they sum up to 1.0. This transformed output represents a probability distribution over the classes, with each neuron outputting the probability of the corresponding class. In our code, the 3 neurons in the output layer correspond to the three classes: 'ham', 'promotional', and 'spam'. The softmax function calculates the likelihood of each class given the input, allowing the model to make predictions based on the class with the highest probability. The categorical cross-entropy loss function, which evaluates the dissimilarity between the predicted probability distribution and the real distribution (one-hot encoded labels), is commonly used in conjunction with softmax activation. During training, the model learns to reduce cross-entropy loss by modifying its parameters to enhance agreement between projected probability and actual labels. So, the softmax activation function in the output layer of the given code makes multi-class classification easier by transforming raw scores into a probability distribution over classes, allowing the model to make confident predictions.

### 4.4.9 Parameters Calculation

#### 1. Embedding Layer

Output Shape: (*None*, 100, 300)

Calculation:

$$\begin{cases} \text{Total Parameters} = \text{vocab\_size} \times \text{embedding\_dim} \\ \text{Total Parameters} = 25000 \times 300 = 7500000 \end{cases}$$

**2. Bidirectional LSTM Layer 1**Output Shape: (*None*, 100, 256)

Calculation:

$$\begin{cases} \text{Total Parameters} = 4 \times ((\text{input dim} + \text{units}) \times \text{units}) \times 2 \\ \text{Total Parameters} = 4 \times ((300 + 128) \times 128) \times 2 = 439296 \end{cases}$$

**3. Dropout Layer**Output Shape: (*None*, 100, 256)

$$\begin{cases} \text{Total Parameters:} & 0 \end{cases}$$

**4. Bidirectional LSTM Layer 2**Output Shape: (*None*, 256)

Calculation:

$$\begin{cases} \text{Total Parameters} = 4 \times ((\text{input dim} + \text{units}) \times \text{units}) \times 2 \\ \text{Total Parameters} = 4 \times ((256 + 128) \times 128) \times 2 = 394240 \end{cases}$$

**5. Dense Layers 1**Output Shape: (*None*, 256)

Calculation:

$$\begin{cases} \text{Total Parameters} = \text{input dim} \times \text{units} + \text{units} \\ \text{Total Parameters} = 256 \times 50 + 50 = 12850 \end{cases}$$

**6. Dense Layer 2**Output Shape: (*None*, 50)

Calculation:

$$\begin{cases} \text{Total Parameters} = \text{input dim} \times \text{units} + \text{units} \\ \text{Total Parameters} = 50 \times 50 + 50 = 2250 \end{cases}$$

**7. Flatten Layer**

Output Shape:  $(None, 50)$

{ Total Parameters: 0

### 8. Dense Layer 3

Output Shape:  $(None, 100)$

Calculation:

$$\begin{cases} \text{Total Parameters} = \text{input dim} \times \text{units} + \text{units} \\ \text{Total Parameters} = 50 \times 100 + 100 = 5150 \end{cases}$$

### 9. Dense Layer 4:

Output Shape:  $(None, 3)$

Calculation:

$$\begin{cases} \text{Total Parameters} = \text{input dim} \times \text{units} + \text{units} \\ \text{Total Parameters} = 100 \times 3 + 3 = 303 \end{cases}$$

10. **Total Parameters:** Total Parameters = Sum of parameters from all layers

$$\text{Total Parameters} = 7500000 + 439296 + 0 + 394240 + 12850 + 2550 + 0 + 5100 + 303 = 8354339$$

## 4.5 Implementation Of Our Machine Learning Model

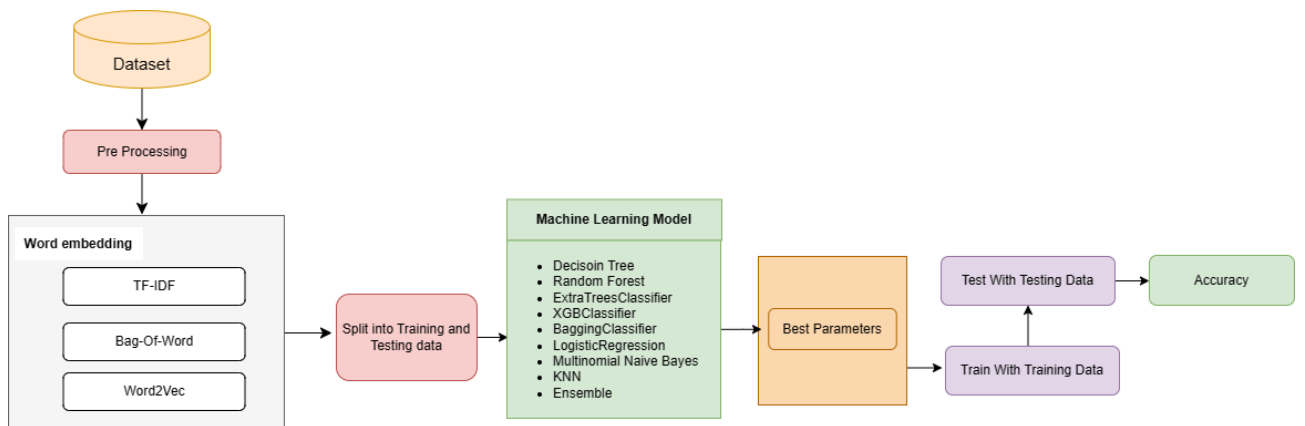


Figure 4.4. Workflow of our Machine Learning Model



It is the final step of our work. After feature extraction the key part is to apply machine learning and deep learning algorithms. These models learn from the features. And we have extracted features from the data. We have implemented Machine Learning and deep learning to perform our task. KNN, Decision Tree, SVM, RandomForest, AdaBoost, LogisticRegression, BaggingClassifier, and ExtraTreesClassifier, XGBClassifier are used as models to find the best accuracy.

A variation of the standard Naïve Bayes classifier, the MNB classifier is frequently applied to various text classification tasks. The KNN is a straightforward, sluggish, instance-based supervised machine learning method that is easy to construct. KNN is an additional option for the regression. The best classifier for binary classifications is a decision tree. It creates decision trees for both classification and regression, using hierarchical attribute usage to determine the sample labels (leaf labels). A maximum margin classifier, or SVM, produces an ideal hyperplane that is used to categorize new samples. In order to categorize a new instance, it employs the Random Forest decision rule, which is composed of several random decision trees that each provide a classification for the incoming data. To classify data, AdaBoost combines a number of weak classifiers into a single strong classifier.

#### 4.5.1 Best performing Ensemble Model

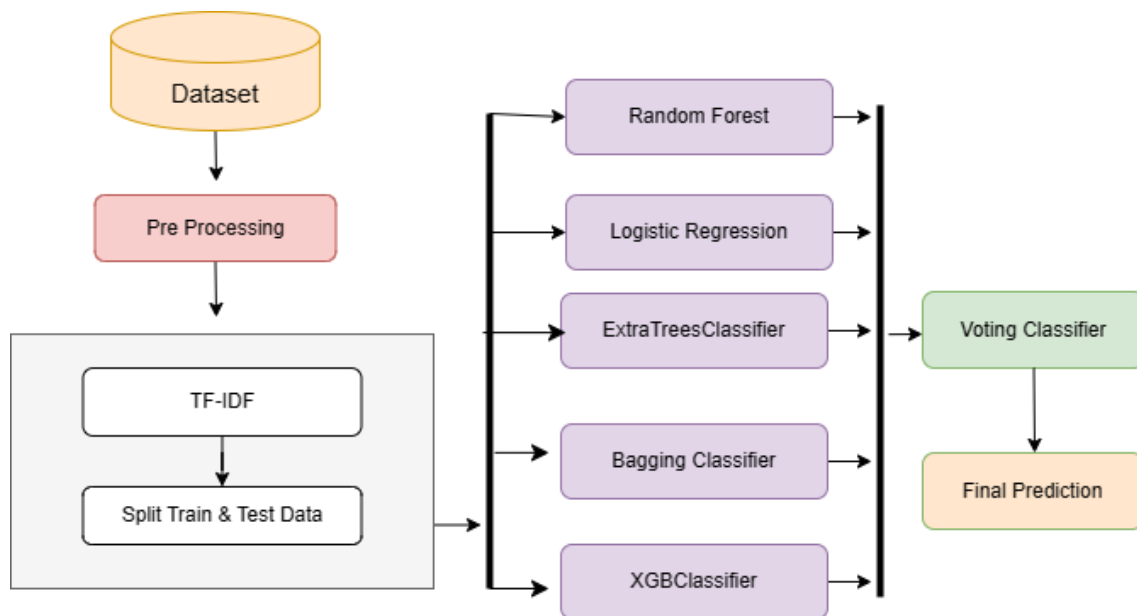


Figure 4.5. Ensemble Model (Voting Classifier)

We created an ensemble model that combines multiple distinct machine learning techniques, each with their own strengths and drawbacks. This ensemble consists of Extra Trees Classifier, Random Forest Classifier, XGB Classifier, Bagging Classifier, Logistic

Regression, SVC, Gradient Boosting Classifier, KNeighbors Classifier, MultinomialNB, AdaBoost Classifier, and Decision Tree Classifier. To create this ensemble, we employ a voting and stacking classifier. In our study, the voteing classifier produced the best results using a machine learning technique. The VotingClassifier generates a final ensemble model that makes predictions using 'soft' voting. In 'soft' voting, the prediction is made using the averaged probabilities calculated by the separate classifiers. The ensemble model is trained using the fit technique on the training data. The training data's labels are transformed to a 1D array using the argmax function. The ensemble model then makes predictions on the test data using the predict approach. Finally, the model's correctness is assessed by comparing the predicted labels to the actual labels in the test data. The ensemble technique uses the strengths of numerous classifiers to produce aggregate predictions, which may result in greater performance than individual models, particularly in situations where distinct models excel at capturing different features of the data.

## CHAPTER 5

## EXPERIMENTAL RESULTS

### 5.1 Overview

This chapter is the result of our attempts to classify Bangla spam emails, and it illustrates the outcomes of using different machine learning and deep learning models. After extensive testing, we were able to validate which is better models' efficacy in separating spam from authentic emails with high accuracy. In our project we have 3 class like spam, ham and promotional. We use in traditional nine different type of algorithm also ensemble with voting and stacking classifier. In our machine learning part we gain 89% for using ensemble model with soft voting classifier . After that we build a Bi LSTM based deep learning model. So deep learning model gain more accuracy. We run our model in epochs=50 and batch size is define 64. That time we gain 99% of our model accuracy and 93% valid accuracy. The chapter is structured to facilitate readers' comprehension o

### 5.2 Hardware Study

We had been leveraging Python 3.7 for various tasks such as data preparation, model development, and evaluation. I've found immense value in libraries like Pandas, NumPy, scikitlearn, and matplotlib. For my deep learning models, we have been using Tensorflow v2.0.0, Bi LSTM & CNN . I've had the chance to work with Tesla T4 GPUs and CUDNNL-STM layers, which have greatly optimized performance in a GPU environment. All this hardware was accessible thanks to Google's Collaboratory, a free Python environment that operates entirely in the cloud.

## 5.3 Performance Metrics

Performance metrics are useful when evaluating the precision and dependability of different techniques used to distinguish between spam (unwanted or unsolicited) and valid emails in the framework of spam email detection. For the purpose to evaluate how well existing algorithms identified Bangla Spam emails, a number of performance parameters was used. These metrics provide visibility into the models' overall efficiency across a variety of areas and also their recall, accuracy, and precision in classification. We applied the next few performance metrics:

**Precision** measures how accurate the model's positive predictions are by calculating the ratio of correctly predicted positive instances to all predicted positive instances.[22] It tells us the proportion of correctly identified positive cases out of all cases predicted as positive. Its Formula is:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \quad (5.1)$$

**Recall** measures the ability of a model to correctly identify all relevant instances, indicating the proportion of true positives among all actual positives.[22] It is calculated as the ratio of true positives to the sum of true positives and false negatives

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \quad (5.2)$$

**F1-score** is the harmonic mean of precision and recall, providing a balance between the two metrics.[22] It quantifies the model's accuracy by considering both false positives and false negatives. F1-score is calculated using the formula:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (5.3)$$

**Accuracy** measures the overall correctness of the model's predictions by calculating the ratio of correctly classified instances to the total number of instances [22]. It provides an indication of how often the model is correct across all classes.

$$\text{Accuracy} = \frac{\text{True Positives (TP)} + \text{True Negatives (TN)}}{\text{Total Observations}} \quad (5.4)$$

**Confusion matrix** provides a detailed overview of a classification model's performance on test data, showing both accurate and inaccurate predictions. It helps assess recall, accuracy, precision, and overall effectiveness in classifying instances. By analyzing true positives, true negatives, false positives, and false negatives, it offers insights beyond simple accuracy metrics, especially in datasets with unequal class distributions.[23]

|        | Predicted      |                      |                      |                      |
|--------|----------------|----------------------|----------------------|----------------------|
|        |                | Ham (H)              | Spam (S)             | Promotional(P)       |
| Actual | Ham (H)        | True Positive (H)    | False Negative (H/S) | False Negative (H/P) |
|        | Spam (S)       | False Negative (S/H) | True Positive (S)    | False Negative (S/P) |
|        | Promotional(P) | False Negative (P/H) | False Negative (P/S) | True Positive (P)    |

## 5.4 Deep learning Approach

### 5.4.1 Text Classification Model with Bi LSTM

Our project uses a BI-LSTM-based deep learning Model. It integrates many layers, such as dense layer, flattened layer, and Bidirectional LSTM networks, a form of Recurrent Neural Network (RNN). We are developing a deep-learning model using TensorFlow and Keras. The model is configured to operate on a GPU device to accelerate processing. It begins with an embedding layer, which turns input data into dense vectors of a specific size.

Next, a Bidirectional LSTM layer of 128 units is added. Now, two dense layers of 50 units each are added, as well as 'relu' activation. Now again uses the BI-LSTM layer of 128 units. Additionally, a Flatten layer is created to flatten the input. Another dense layer of 100 units and L2 regularization is added. The last layer is a dense layer with three units and softmax activation, which is commonly employed in multi-class classification. The model is built using the Adam optimizer and the categorical cross-entropy for loss function. Given the usage of Embedding and LSTM layers, it appears that our BI-LSTM based deep learning model was built for a text categorization problem. The specific job would be determined by the data and how it was processed before to being put into the model. Our model's architecture and hyperparameters may need to be modified depending on the specific task and data.

In Our BI-LSTM based deep learning model we use is batch size is 64 and epochs is 60 this time we gain the highest number of accuracy is 99.97% and we find the test accuracy

Table 5.1. BI-LSTM Based Model Performance Metrics

| Epochs & Batch Size | Category    | Precision | Recall | F1-Score | Accuracy (%) |
|---------------------|-------------|-----------|--------|----------|--------------|
| 50 & 64             | Ham         | 0.92      | 0.91   | 0.92     | 92.24%       |
|                     | Spam        | 0.97      | 0.95   | 0.96     |              |
|                     | Promotional | 0.91      | 0.93   | 0.92     |              |
| 60 & 64             | Ham         | 0.92      | 0.92   | 0.92     | 94.46%       |
|                     | Spam        | 0.98      | 0.95   | 0.96     |              |
|                     | Promotional | 0.91      | 0.93   | 0.92     |              |
| 60 & 120            | Ham         | 0.90      | 0.93   | 0.92     | 93.77%       |
|                     | Spam        | 0.98      | 0.96   | 0.97     |              |
|                     | Promotional | 0.92      | 0.91   | 0.92     |              |
| 65 & 64             | Ham         | 0.91      | 0.91   | 0.91     | 93.50%       |
|                     | Spam        | 0.98      | 0.94   | 0.96     |              |
|                     | Promotional | 0.89      | 0.94   | 0.91     |              |

is 94.46%. In the table, we also share some parameter changing accuracy results. So we try lots of changing the parameters in our model but this is the highest. In the upper below table 5.1 we show some of our model result, In the below we are showing the confusion matrix based on our highest accuracy perimeter.

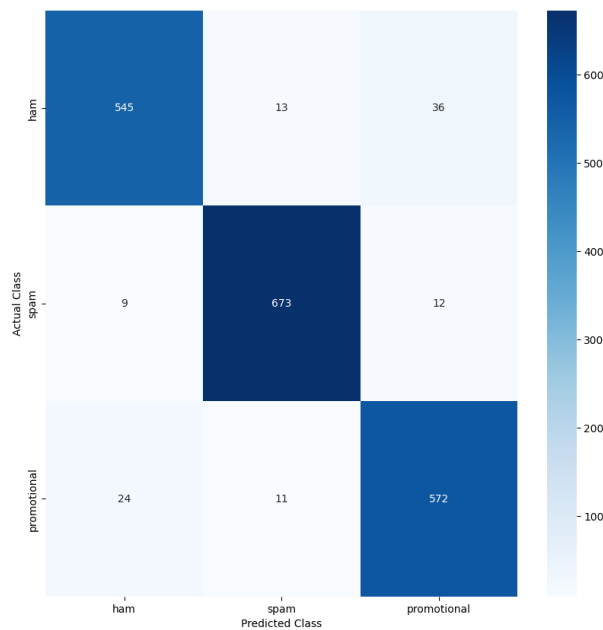


Figure 5.1. Confusion matrix of Bi-LSTM Based Model

#### 5.4.1.1 Accuracy vs Validation Accuracy

Accuracy measures the proportion of correct predictions made by your model on the training data. The validation Accuracy is defined as evaluates the model's performance on a

separate validation dataset that the model hasn't been trained on. And the accuracy curve, a rising line, depicts the model's performance with the training data. It demonstrates how well the model learns and adapts to the patterns in the training dataset. The validation accuracy curve, which frequently shadows the accuracy curve, displays the model's performance on unseen data. If these two lines begin to diverge, it may indicate overfitting, in which the model performs well on training data but not on fresh, unknown data.

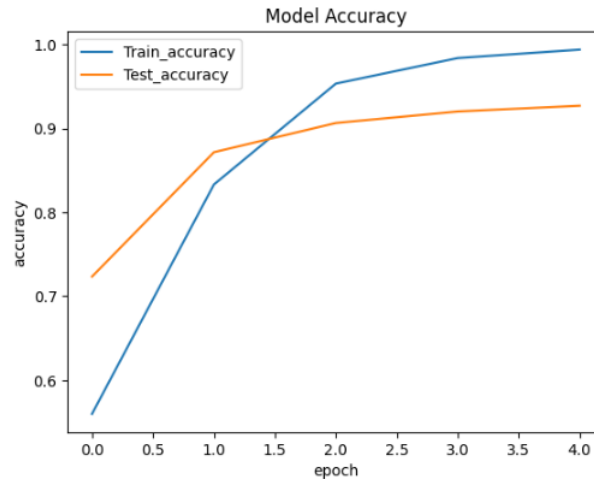


Figure 5.2. Accuracy & Valid Accuracy Curve

The figure 5.2 visualizes the training and validation accuracy of a neural network model over epochs, providing insights into how well the model is learning from the training data and generalizing to unseen data. The model accuracy starts at around 0.6 and increases to nearly 1.0 by the end of training. The validation accuracy starts at around 0.5 and increases to around 0.8 by the end of training. This means the model is performing well on the training data but may not perform as well on unseen data. In our Bi-LSTM model we gain a maximum train accuracy is 99.97 % and a test accuracy is gain 94.46 % for 60 epochs 64 batch size.

#### 5.4.1.2 Loss vs Validation Loss

Loss function that quantifies the difference between the model's predictions and the true labels. The training process aims to minimize this loss function. And validation Loss calculated on the validation dataset. It helps monitor how well the model is learning without overfitting to the training data. A consistently decreasing validation loss suggests the model is improving, while a sudden increase might indicate overfitting. The loss curve shows the model's training error, decreasing as the model learns. The validation loss curve tracks error on unseen data. Divergence of these curves may indicate overfitting.

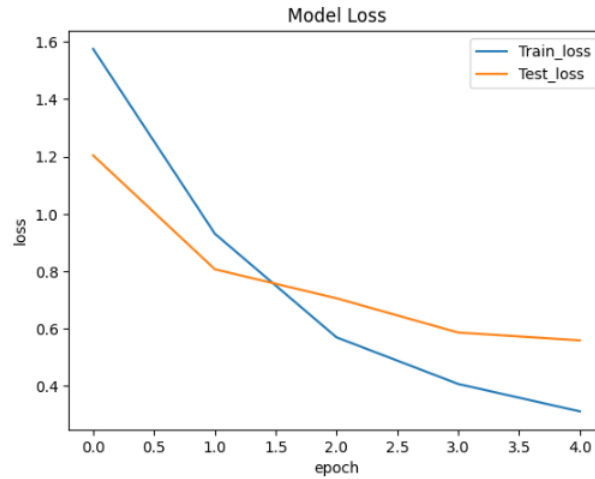


Figure 5.3. Loss &amp; Valid Loss Curve

The figure 5.3 shows a line graph of the model loss during training on a recurrent neural network (RNN). The x-axis represents the epoch, which is one complete pass through the training data. The y-axis represents the loss, which is a measure of how well the model performs on a given set of data. The blue line represents the training loss, which is the loss calculated on the training data. The red line represents the validation loss, which is the loss calculated on a separate set of data that the model has not been trained on. The validation loss is used to monitor how well the model is generalizing to unseen data. In our model, the training loss starts at around 1.4 and decreases to around 0.4 by the end of training. The validation loss starts at around 1.2 and decreases to around 0.6 by the end of training. The model appears to have converged by the end of training, as the loss is no longer decreasing significantly. Overall, the figure shows that the model is learning well and generalizing well to unseen data.

#### 5.4.1.3 Receiver operating characteristic curve(ROC)

The ROC curve is a graphical plot that illustrates the performance of a binary classification model across different threshold settings. It plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at various threshold settings. TPR represents the proportion of actual positive cases that were correctly identified as positive (also known as sensitivity or recall), while FPR represents the proportion of actual negative cases that were incorrectly classified as positive.



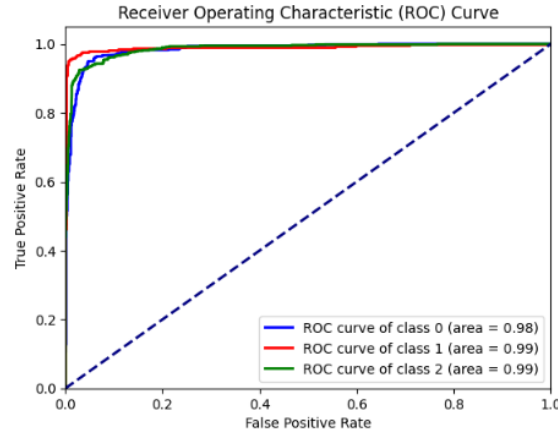


Figure 5.4. ROC Curve

It is a graphical diagram that shows how well a binary classifier model performs at different threshold levels. Plotting the True Positive Rate (TPR) against the False Positive Rate (FPR) for three classes is the ROC curve shown in the figure. TPR, also known as sensitivity, is the percentage of positive cases that the model properly recognized. FPR is the percentage of negative cases that the model mistakenly classified as positive. A model with a higher TPR and a lower FPR is said to be performing better when its ROC curve is closer to the top-left corner. The area under the ROC curve (AUC) is a numerical measure of a classifier's performance. A larger AUC indicates a better classifier. In the figure, the ROC curve of class 0 has an area of 0.97, the ROC curve of class 1 has an area of 0.99 and the ROC curve of class 2 has an area of 0.97.

## 5.5 Machine Learning Approach

### 5.5.1 Extra Trees Classifier

Our First Algorithm is the Extra Trees Classifier. Our train dataset is 80% and test dataset is 20%. We find the best accuracy of 88.971% with respect to the parameter  $n$  estimators=600, random state=15 by using the word embedding technique. For this, we find the classification report which is given below:

The machine learning performance evaluation metrics, which are used to show the precision, recall and F1 Score of our trained classification model, are briefly described in TABLE 5.2. The remaining parameter adjustment provided an inefficient outcome because the accuracy shifted a little.

Table 5.2. Extra Trees Classifier Performance Metrics

| Word Embedding | Category    | Precision | Recall | F1-Score | Accuracy (%) |
|----------------|-------------|-----------|--------|----------|--------------|
| Tf-IDF         | Ham         | 0.90      | 0.85   | 0.88     | 88.97%       |
|                | Spam        | 0.91      | 0.80   | 0.85     |              |
|                | Promotional | 0.98      | 0.88   | 0.93     |              |
| BOW            | Ham         | 0.89      | 0.84   | 0.86     | 88.34%       |
|                | Spam        | 0.88      | 0.79   | 0.83     |              |
|                | Promotional | 0.97      | 0.87   | 0.92     |              |
| Word2vec       | Ham         | 0.79      | 0.67   | 0.73     | 79.94%       |
|                | Spam        | 0.84      | 0.68   | 0.75     |              |
|                | Promotional | 0.90      | 0.87   | 0.88     |              |

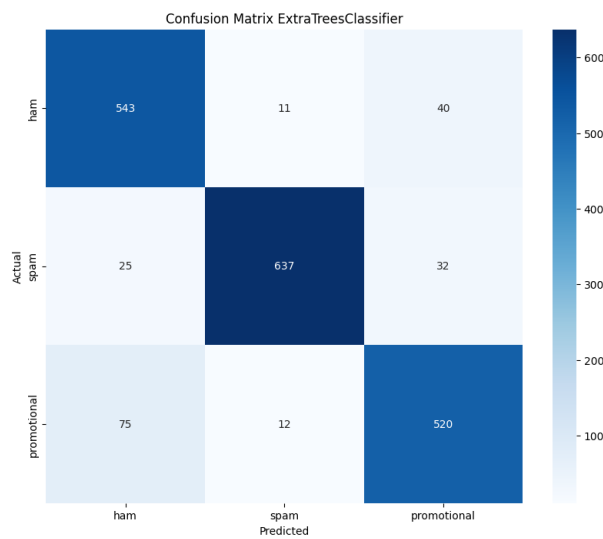


Figure 5.5. Confusion matrix of Extra Trees Classifier

### 5.5.2 Random Forest Classifier

Then, our applied Algorithm is Extra Trees Classifier. We find the best accuracy 87.2823% with respect to the parameter  $n_{\text{estimators}}=100$ ,  $\text{random\_state}=42$  by using the word embedding technique of TF-IDF. For this we find the classification report which is given below:

The machine learning performance evaluation metrics, which are used to show the precision, recall and F1 Score of our trained classification model, are briefly described in TABLE 5.3. The remaining parameter adjustment provided an inefficient outcome because the accuracy shifted a little.

Table 5.3. Random Forest Performance Metrics

| Word Embedding | Category    | Precision | Recall | F1-Score | Accuracy (%) |
|----------------|-------------|-----------|--------|----------|--------------|
| Tf-IDF         | Ham         | 0.90      | 0.79   | 0.84     | 87.28%       |
|                | Spam        | 0.87      | 0.79   | 0.83     |              |
|                | Promotional | 0.98      | 0.90   | 0.94     |              |
| BOW            | Ham         | 0.88      | 0.79   | 0.83     | 81.63%       |
|                | Spam        | 0.85      | 0.79   | 0.82     |              |
|                | Promotional | 0.97      | 0.87   | 0.91     |              |
| Word2vec       | Ham         | 0.78      | 0.67   | 0.72     | 79.94%       |
|                | Spam        | 0.84      | 0.66   | 0.73     |              |
|                | Promotional | 0.91      | 0.86   | 0.88     |              |

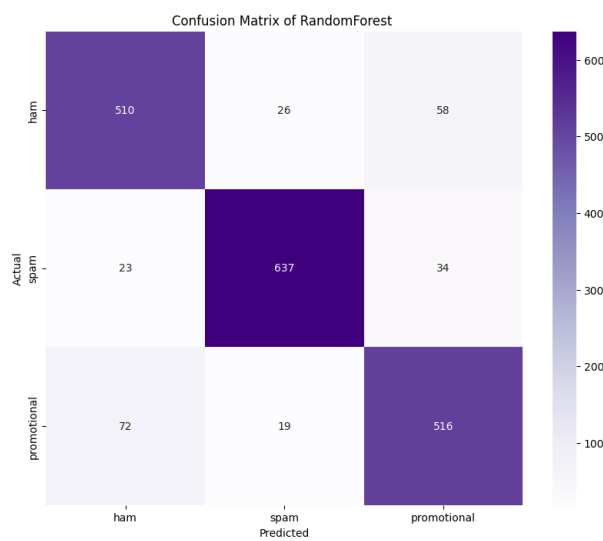


Figure 5.6. Confusion matrix of Random Fores

### 5.5.3 Extreme Gradient Boosting Classifier

Then, The Algorithm is an Extreme Gradient Boosting Classifier. We find the best accuracy 85.3298% with respect to the parameter  $n$  estimators=100, random state=42. For this we find the classification report which is given below:

Table 5.4 This table facilitates a thorough assessment of the eXtreme Gradient Boosting's performance across several categories and embedding techniques by giving precision, recall, F1-score, and accuracy for each category and word embedding type

Table 5.4. Extreme Gradient Boosting Classifier Performance Metrics

| Word Embedding | Category    | Precision | Recall | F1-Score | Accuracy (%) |
|----------------|-------------|-----------|--------|----------|--------------|
| Tf-IDF         | Ham         | 0.86      | 0.79   | 0.83     | 85.33%       |
|                | Spam        | 0.84      | 0.85   | 0.84     |              |
|                | Promotional | 0.96      | 0.88   | 0.92     |              |
| BOW            | Ham         | 0.87      | 0.72   | 0.79     | 84.96%       |
|                | Spam        | 0.80      | 0.83   | 0.81     |              |
|                | Promotional | 0.95      | 0.88   | 0.92     |              |
| Word2vec       | Ham         | 0.78      | 0.71   | 0.74     | 80.99%       |
|                | Spam        | 0.81      | 0.69   | 0.75     |              |
|                | Promotional | 0.90      | 0.88   | 0.89     |              |

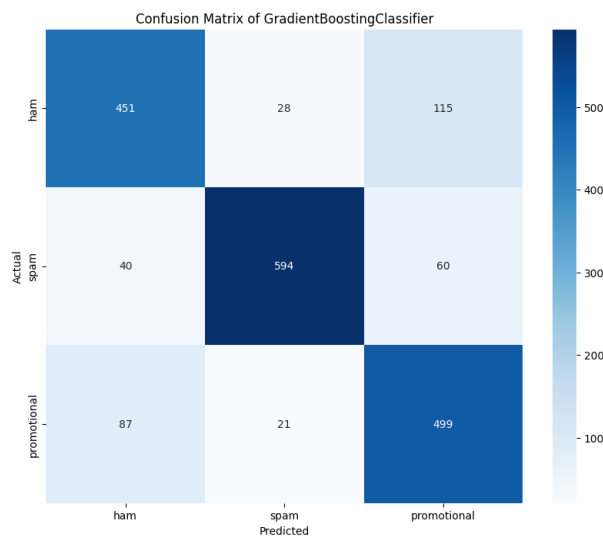


Figure 5.7. Confusion matrix of Extreme Gradient Boosting

### 5.5.4 Bagging Classifier Classifier

After that, we tried the Algorithm which is Bagging Classifier. We find the best accuracy of 83.95% with respect to the parameter  $n_{\text{estimators}}=60$ ,  $\text{random\_state}=15$  by using word embedding technique of TF-IDF. For this we find the classification report which is given below:

The machine learning performance evaluation metrics, which are used to show the precision, recall and F1 Score. of our trained classification model are briefly described in TABLE 5.5. The remaining parameter adjustment provided an inefficient outcome because the accuracy shifted a little.

Table 5.5. Bagging Classifier Performance Metrics

| Word Embedding | Category    | Precision | Recall | F1-Score | Accuracy (%) |
|----------------|-------------|-----------|--------|----------|--------------|
| Tf-IDF         | Ham         | 0.79      | 0.82   | 0.81     | 83.95%       |
|                | Spam        | 0.84      | 0.82   | 0.83     |              |
|                | Promotional | 0.90      | 0.89   | 0.90     |              |
| BOW            | Ham         | 0.80      | 0.83   | 0.81     | 83.21%       |
|                | Spam        | 0.80      | 0.77   | 0.79     |              |
|                | Promotional | 0.89      | 0.89   | 0.89     |              |
| Word2vec       | Ham         | 0.74      | 0.78   | 0.76     | 80.15%       |
|                | Spam        | 0.79      | 0.72   | 0.75     |              |
|                | Promotional | 0.88      | 0.90   | 0.89     |              |

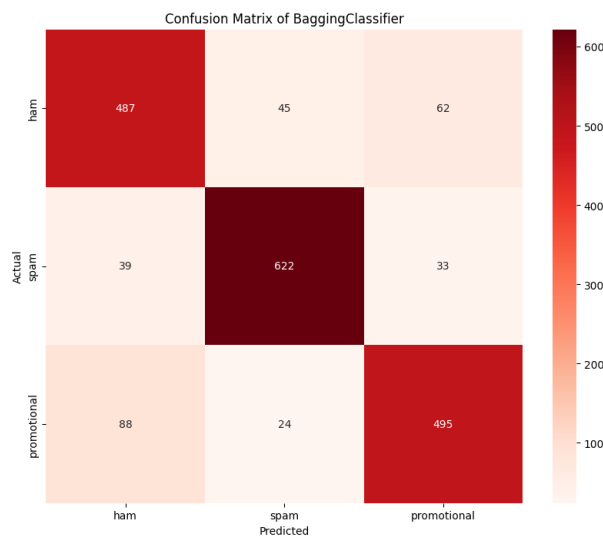


Figure 5.8. Confusion matrix of Bagging Classifier

### 5.5.5 Logistic Regression

After that we tried the Algorithm which is Logistic Regression. We find the best accuracy 54.16% with respect to the parameter solver='liblinear', penalty='l1' by using word embedding technique of BOW .For this we find the classification report which is given below:

The machine learning performance evaluation metrics, which are used to show the precision, recall and F1 Score. of our trained classification model are briefly described in TABLE 5.6 .The remaining parameter adjustment provided an inefficient outcome because the accuracy shifted a little.

Table 5.6. Logistic Regression Performance Metrics

| Word Embedding | Category    | Precision | Recall | F1-Score | Accuracy (%) |
|----------------|-------------|-----------|--------|----------|--------------|
| Tf-IDF         | Ham         | 0.81      | 0.80   | 0.80     | 82.90%       |
|                | Spam        | 0.79      | 0.83   | 0.81     |              |
|                | Promotional | 0.91      | 0.89   | 0.90     |              |
| BOW            | Ham         | 0.84      | 0.75   | 0.79     | 84.16%       |
|                | Spam        | 0.76      | 0.87   | 0.81     |              |
|                | Promotional | 0.93      | 0.89   | 0.91     |              |
| Word2vec       | Ham         | 0.74      | 0.67   | 0.70     | 73.12%       |
|                | Spam        | 0.68      | 0.71   | 0.69     |              |
|                | Promotional | 0.81      | 0.85   | 0.83     |              |

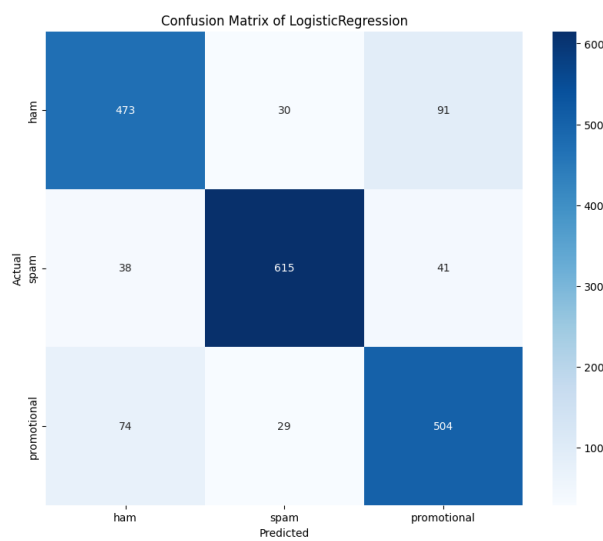


Figure 5.9. Confusion matrix of Logistic Regression

### 5.5.6 Decision Tree Classifier

Again we tried with the Algorithm which is Decision Tree. We find the best accuracy 63.1662% with respect to the parameter max depth=5 by using word embedding technique of TF-IDF. For this, we find the classification report, which is given below:

The machine learning performance evaluation metrics, which are used to show the precision, recall and F1 Score. of our trained classification model are briefly described in TABLE 5.7. The additional parameter adjustments yielded marginal improvements in accuracy, failing to produce a substantial shift in performance.

Table 5.7. Decision Tree Classifier Performance Metrics

| Word Embedding | Category    | Precision | Recall | F1-Score | Accuracy (%) |
|----------------|-------------|-----------|--------|----------|--------------|
| Tf-IDF         | Ham         | 0.68      | 0.40   | 0.51     | 63.16%       |
|                | Spam        | 0.53      | 0.82   | 0.65     |              |
|                | Promotional | 0.83      | 0.72   | 0.77     |              |
| BOW            | Ham         | 0.68      | 0.35   | 0.46     | 63.06%       |
|                | Spam        | 0.49      | 0.80   | 0.60     |              |
|                | Promotional | 0.83      | 0.74   | 0.78     |              |
| Word2vec       | Ham         | 0.67      | 0.64   | 0.66     | 69.03%       |
|                | Spam        | 0.65      | 0.64   | 0.64     |              |
|                | Promotional | 0.76      | 0.80   | 0.78     |              |

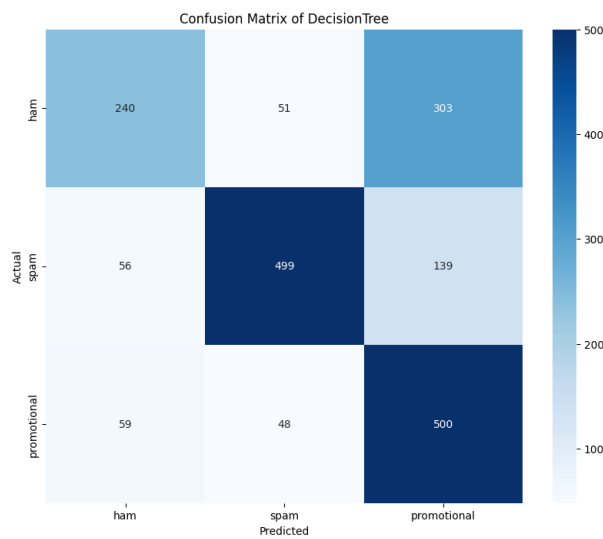


Figure 5.10. Confusion matrix of Decision Tree

### 5.5.7 Support Vector Machine

Again we tried with the Algorithm which is Support Vector Machine. We find the best accuracy 82.4802% with respect to the parameter kernel='sigmoid', gamma=1.0, probability=True word embedding technique of TF-IDF. For this we find the classification report which is given below-

TABLE 5.8 provides essential details regarding the metrics used to assess machine learning effectiveness, including precision, recall, and F1Score for the classification model we developed. The parameter changes resulted in only slight accuracy improvements, with little impact on total performance.

Table 5.8. Support Vector Machine Classifier Performance Metrics

| Word Embedding | Category    | Precision | Recall | F1-Score | Accuracy (%) |
|----------------|-------------|-----------|--------|----------|--------------|
| Tf-IDF         | Ham         | 0.78      | 0.82   | 0.80     | 82.48%       |
|                | Spam        | 0.80      | 0.79   | 0.79     |              |
|                | Promotional | 0.91      | 0.88   | 0.89     |              |
| BOW            | Ham         | 0.49      | 0.32   | 0.39     | 43.21%       |
|                | Spam        | 0.44      | 0.45   | 0.44     |              |
|                | Promotional | 0.41      | 0.51   | 0.45     |              |
| Word2vec       | Ham         | 0.23      | 0.29   | 0.26     | 25.75%       |
|                | Spam        | 1.00      | 0.02   | 0.05     |              |
|                | Promotional | 0.26      | 0.40   | 0.31     |              |

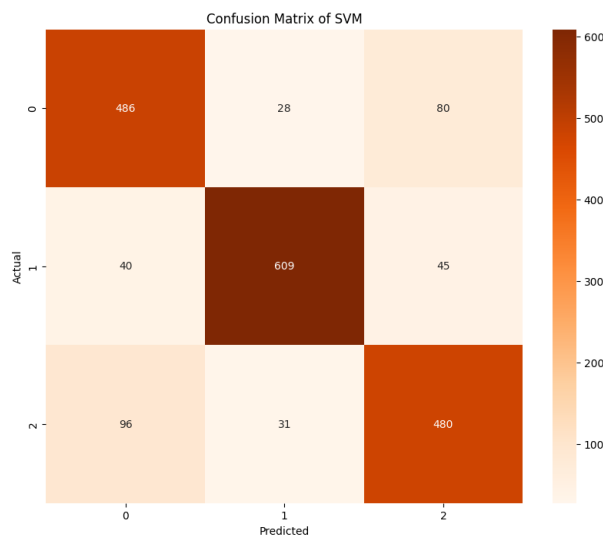


Figure 5.11. Confusion matrix of Support Vector Machine

### 5.5.8 K-Nearest Neighbor Classifier

Again we tried with the Algorithm which is K-nearest Algorithm. We find the accuracy 73.08% with respect to the parameter of k value is 3 by using word embedding technique of BOW. For this, we find the classification report which is given below-

TABLE 5.9 provides essential details regarding the metrics used to assess machine learning effectiveness, including precision, recall, and F1Score for the classification model we developed.



Table 5.9. K-Nearest Neighbor Performance Metrics

| Word Embedding | Category    | Precision | Recall | F1-Score | Accuracy (%) |
|----------------|-------------|-----------|--------|----------|--------------|
| Tf-IDF         | Ham         | 0.85      | 0.40   | 0.54     | 58.10%       |
|                | Spam        | 0.44      | 0.91   | 0.59     |              |
|                | Promotional | 0.95      | 0.45   | 0.61     |              |
| BOW            | Ham         | 0.81      | 0.57   | 0.69     | 72.40%       |
|                | Spam        | 0.61      | 0.91   | 0.73     |              |
|                | Promotional | 0.94      | 0.70   | 0.80     |              |
| Word2vec       | Ham         | 0.74      | 0.64   | 0.68     | 73.08%       |
|                | Spam        | 0.73      | 0.66   | 0.70     |              |
|                | Promotional | 0.79      | 0.87   | 0.83     |              |

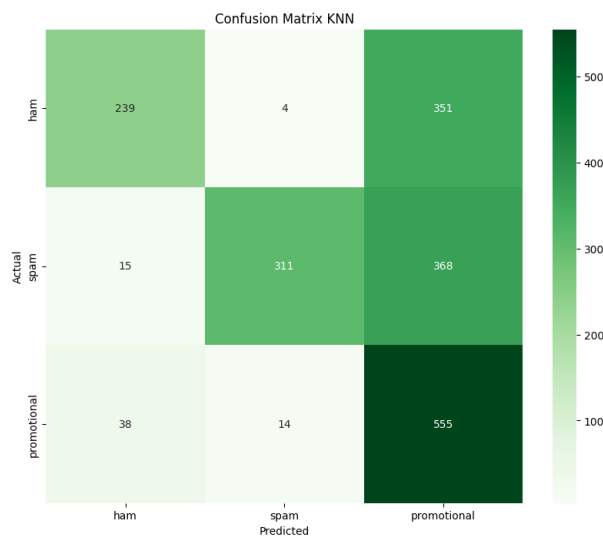


Figure 5.12. Confusion matrix of K-Nearest Neighbor

### 5.5.9 Multinomial Naive Bayes

Now, we tried with the Algorithm which is Multinomial Naive Bayes. We find the accuracy 75.88% with respect to the parameter by using word embedding technique BOW. For this, we find the classification report which is given below-

Table 5.10. Multinomial Naive Bayes Performance Metrics

| Word Embedding | Category    | Precision | Recall | F1-Score | Accuracy (%) |
|----------------|-------------|-----------|--------|----------|--------------|
| Tf-IDF         | Ham         | 0.77      | 0.78   | 0.78     | 75.21%       |
|                | Spam        | 0.86      | 0.53   | 0.66     |              |
|                | Promotional | 0.70      | 0.93   | 0.80     |              |
| BOW            | Ham         | 0.78      | 0.72   | 0.75     | 75.88%       |
|                | Spam        | 0.79      | 0.60   | 0.69     |              |
|                | Promotional | 0.73      | 0.92   | 0.82     |              |

TABLE 5.10 provides essential details regarding the metrics used to assess machine learning effectiveness, including precision, recall, and F1Score for the classification model we developed.

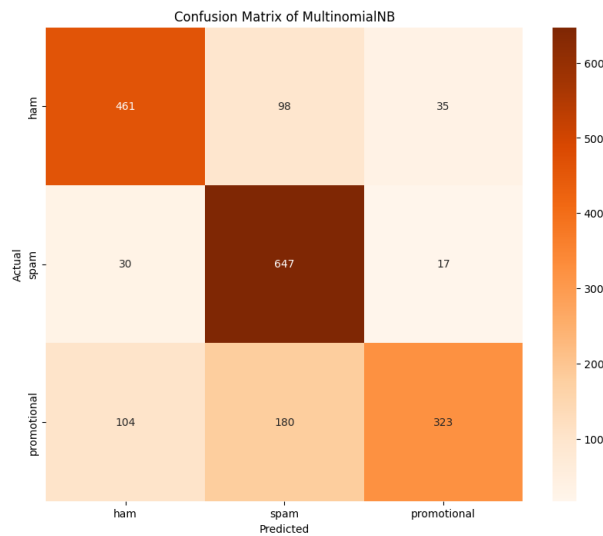


Figure 5.13. Confusion matrix of Multinomial Naive Bayes

### 5.5.10 Ensemble Model

Lastly , We tried Ensemble Model, we found the best accuracy 89.34% with respect to the parameters estimators is [ Multinomial Naive Bayes, ExtraTreesClassifier,Random Forest, Bagging Classifier, XGBClassifier],voting='soft' by using word embedding technique. Here the classification Report is given

The classification report for an ensemble model with voting and stacking ensemble strategies that uses various word embedding approaches (TF-IDF, Bag-of-Words, and Word2Vec) is shown in Table 6.11.By iteratively refining the model and exploring these strategies, it's possible to improve the performance metrics and enhance the overall effectiveness of the classification system. TF-IDF When compared to Bag-of-Words and Word2Vec, embedding consistently produces better results across all categories.In terms of accuracy, voting ensembles typically do better than stacking ensembles.Different categories and embedding procedures yield different scores for precision, recall, and F1-score; promotional messages frequently have the greatest ratings

Table 5.11. Ensemble Model Performance Metrics

| Classifier | Word Embedding | Category    | Precision | Recall | F1-Score | Accuracy (%) |
|------------|----------------|-------------|-----------|--------|----------|--------------|
| Voting     | TF-TDF         | Ham         | 0.86      | 0.89   | 0.87     | 89.34%       |
|            |                | Spam        | 0.88      | 0.85   | 0.87     |              |
|            |                | Promotional | 0.93      | 0.94   | 0.94     |              |
|            | BOW            | Ham         | 0.86      | 0.85   | 0.85     | 87.54%       |
|            |                | Spam        | 0.83      | 0.86   | 0.85     |              |
|            |                | Promotional | 0.93      | 0.91   | 0.92     |              |
|            | Word2Vec       | Ham         | 0.76      | 0.78   | 0.77     | 82.01%       |
|            |                | Spam        | 0.80      | 0.75   | 0.77     |              |
|            |                | Promotional | 0.88      | 0.91   | 0.90     |              |
| Stacking   | TF-TDF         | Ham         | 0.84      | 0.88   | 0.86     | 88.91%       |
|            |                | Spam        | 0.86      | 0.86   | 0.86     |              |
|            |                | Promotional | 0.96      | 0.92   | 0.94     |              |
|            | BOW            | Ham         | 0.87      | 0.86   | 0.87     | 88.70%       |
|            |                | Spam        | 0.83      | 0.88   | 0.85     |              |
|            |                | Promotional | 0.95      | 0.92   | 0.93     |              |
|            | Word2Vec       | Ham         | 0.74      | 0.79   | 0.76     | 78.78%       |
|            |                | Spam        | 0.76      | 0.74   | 0.75     |              |
|            |                | Promotional | 0.90      | 0.87   | 0.88     |              |

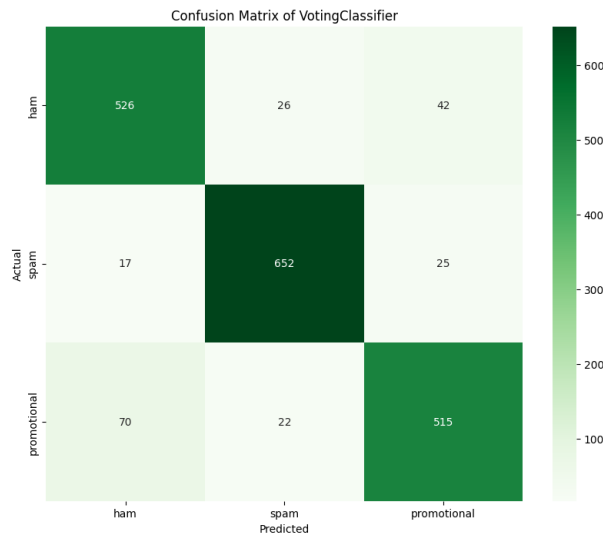


Figure 5.14. Confusion matrix of Ensemble Voting Classifier

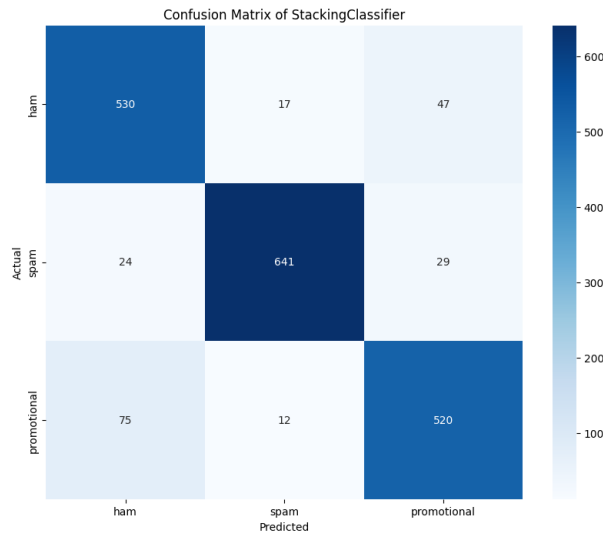


Figure 5.15. Confusion matrix of Ensemble Stacking Classifier

## 5.6 Comparison

Table 5.12. Models Accuracy Comparison

| Model                                | Accuracy(%) |
|--------------------------------------|-------------|
| Logistic Regression                  | 84.16       |
| Extreme Gradient Boosting Classifier | 85.33       |
| Random Forest                        | 87.28       |
| Ensemble Model(Stacking Classifier)  | 88.91       |
| Extra Trees Classifier               | 88.97       |
| Ensemble Model(Voting Classifier)    | 89.34       |
| Bi-LSTM based Model                  | 94.46       |

In this comparison part, we're showing the top 7 performing Machine and Deep learning models based on our own dataset. From Table 5.12 we notice the accuracy of Logistic Regression is 84.16%, the Extreme Gradient Boosting Classifier is 85.33%, Random Forest is 87.28%, Ensemble Model using Stacking Classifier is 88.91%, Extra Trees Classifier is 88.97%, Ensemble Model using Voting Classifier is 89.34% and Bi-LSTM baased deep learning model is 94.46%. If we observe that the Bi-LSTM-based deep learning model outperforms all others with an impressive accuracy. So the better perform in machine learning is the Ensemble model which is based on voting classification and the Bi-LSTM model for our dataset. finally, we got our best model is BI-LSTM based deep learning model.

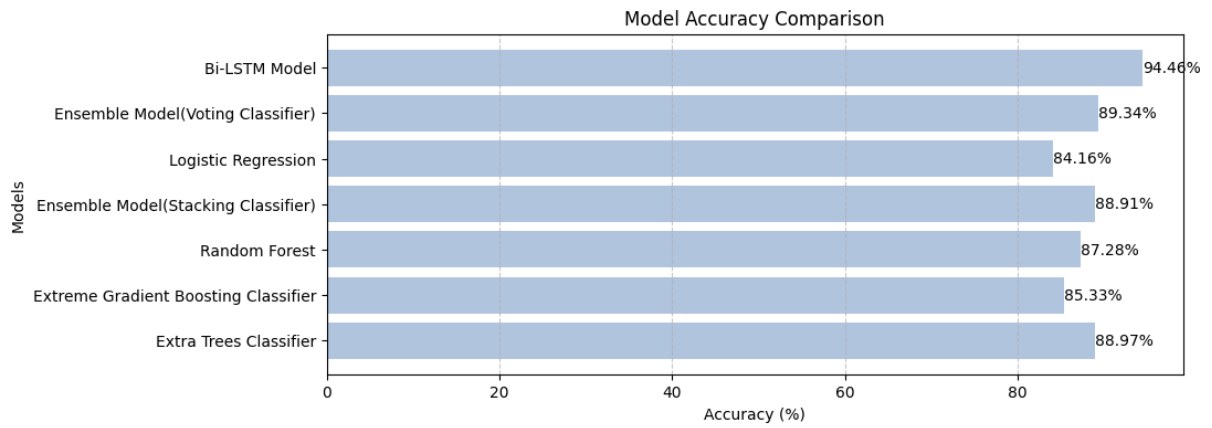


Figure 5.16. Models Accuracy Comparison

## 6.1 Limitations

Despite this research met its aims, there are a few limitations to consider for an in-depth comprehension of the results. Here are some specific limits that should consider:

- **Challenges in analyzing the model's generality:** Evaluating a model's generality entails analyzing how well it performs across multiple datasets or in situations beyond the data from which it was developed. By addressing these challenges through careful data collection, model training, evaluation, and validation techniques, researchers and practitioners can better understand and improve the generality of machine learning models.
- **Localization and Multilingual Support:** Detecting Spam emails accross languages and region is tough due to linguistic and cultural differences. Developing effective models to handle this complexity is difficult yet essential. Different languages have unique grammatical structures, vocabulary, and writing styles. What constitutes spam in one language may not be the same in another. For example, the characteristics of a typical spam email in English, such as exaggerated claims or poor grammar, might not apply to spam in other languages.
- **Real-world Deployment Considerations:** The thesis may not have gone into great detail into the actual issues of establishing spam detection systems in daily life. Model interpretability, latency requirements, integration with current email infrastructure, and user feedback methods are all important for successful implementation,

but they may not have been sufficiently studied

## 6.2 Future Work

The confines of our study give areas for further research: To Enrich the Dataset: The key objective of our future effort is to expand our dataset. Include a greater number of emails to improve the model's resilience and ability to generalize across a broad range of Spam emails and its variants.

- **Enhancing Model Performance:** In order to obtain more accuracy and robustness, machine learning and deep learning models will need to be improved and optimised in future work on Bengali spam email detection. Investigating cutting-edge methods like ensemble learning, transfer learning, and model fine-tuning may be necessary to raise performance indicators like F1-score, recall, and precision. Furthermore, the sustained efficacy of the models will depend on ongoing observation and modification in response to changing spam strategies and user input.
- **Addressing Multilingual Approaches:** As spam emails may be written in multiple languages, including English and Bengali, future research can focus on developing multilingual detection systems capable of identifying and classifying spam across different languages. This involves adapting existing models to handle multilingual text processing, exploring language-agnostic features, and leveraging cross-lingual transfer learning techniques to enhance detection accuracy and scalability across diverse linguistic contexts.
- **Real-Time Detection:** It is essential to detect spam emails in real time in order to quickly identify and mitigate new risks. Future research might focus on creating real-time detection systems that use scalable structures and effective algorithms to continuously scan incoming emails for indications of spam, ensuring prompt detection and response. This could entail using cloud-based infrastructure, parallelizing algorithms, and integrating streaming data processing frameworks to enable scalable and quick spam detection in real-world email systems.

By focusing on these areas of future work, researchers and practitioners can further advance the field of Bengali spam email detection, enhancing model performance, addressing multilingual challenges, and enabling real-time detection to effectively combat evolving spam threats and safeguard users' online security and privacy.

## 6.3 Conclusion

Finally, our research aims to overcome the issues of classifying Bengali emails as spam or ham or promotional by presenting a new Bengali spam email dataset. We conducted extensive experimentation to assess the efficacy of various machine learning models, highlighting the superior performance of ensemble approaches, particularly the ensemble model incorporating Random Forest, Logistic Regression, ExtraTreesClassifier, Bagging Classifier, XGBClassifier which achieved an impressive accuracy of 89.34%. Furthermore, our investigation into deep learning approaches highlighted the effectiveness of a Bi-LSTM model based in detecting spam emails, which achieved a stunning accuracy of 99.97% and a valid accuracy of 94.46%. Our findings highlight the need of using a wide range of machine learning methods, from traditional classifiers to deep learning architectures, in efficiently combatting spam emails. As we look ahead, further research and innovation in machine learning and deep learning will be critical in addressing increasing spam threats and ensuring the accuracy of digital communication channels.



---

## BIBLIOGRAPHY

- [1] R. Zannat, A. A. Mumu, A. R. Khan, T. Mubashshira, and S. R. Mahmud, “A deep learning-based approach for detecting bangla spam emails,” in *2023 3rd International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME)*, IEEE, 2023, pp. 1–6.
- [2] R. Amin, M. M. Rahman, and N. Hossain, “A bangla spam email detection and datasets creation approach based on machine learning algorithms,” in *2019 3rd International Conference on Electrical, Computer & Telecommunication Engineering (ICECTE)*, IEEE, 2019, pp. 169–172.
- [3] M. M. Uddin, M. Yasmin, M. S. H. Khan, M. I. Rahman, and T. Islam, “Detecting bengali spam sms using recurrent neural network.,” *J. Commun.*, vol. 15, no. 4, pp. 325–331, 2020.
- [4] T. Islam, S. Latif, and N. Ahmed, “Using social networks to detect malicious bangla text content,” in *2019 1st International Conference on Advances in Science, Engineering and Robotics Technology (ICASERT)*, IEEE, 2019, pp. 1–4.
- [5] S. Ahammed, M. Rahman, M. H. Niloy, and S. M. H. Chowdhury, “Implementation of machine learning to detect hate speech in bangla language,” in *2019 8th International Conference System Modeling and Advancement in Research Trends (SMART)*, IEEE, 2019, pp. 317–320.
- [6] O. Sen, M. Fuad, M. N. Islam, *et al.*, “Bangla natural language processing: A comprehensive analysis of classical, machine learning, and deep learning-based methods,” *IEEE Access*, vol. 10, pp. 38 999–39 044, 2022.
- [7] L. Rokach and O. Maimon, “Decision trees,” *Data mining and knowledge discovery handbook*, pp. 165–192, 2005.

- [8] L. Breiman, “Random forests,” *Machine learning*, vol. 45, pp. 5–32, 2001.
- [9] T. Evgeniou and M. Pontil, “Support vector machines: Theory and applications,” in *Advanced course on artificial intelligence*, Springer, 1999, pp. 249–257.
- [10] A. M. Kibriya, E. Frank, B. Pfahringer, and G. Holmes, “Multinomial naive bayes for text categorization revisited,” in *AI 2004: Advances in Artificial Intelligence: 17th Australian Joint Conference on Artificial Intelligence, Cairns, Australia, December 4-6, 2004. Proceedings 17*, Springer, 2005, pp. 488–499.
- [11] J. S. Cramer, “The origins of logistic regression,” 2002.
- [12] Z. Zhang, “Introduction to machine learning: K-nearest neighbors,” *Annals of translational medicine*, vol. 4, no. 11, 2016.
- [13] A. Mohammed and R. Kora, “A comprehensive review on ensemble deep learning: Opportunities and challenges,” *Journal of King Saud University-Computer and Information Sciences*, vol. 35, no. 2, pp. 757–774, 2023.
- [14] R. E. Schapire, “Explaining adaboost,” in *Empirical Inference: Festschrift in Honor of Vladimir N. Vapnik*, Springer, 2013, pp. 37–52.
- [15] L. Breiman, “Bagging predictors,” *Machine learning*, vol. 24, pp. 123–140, 1996.
- [16] P. Geurts, D. Ernst, and L. Wehenkel, “Extremely randomized trees,” *Machine learning*, vol. 63, pp. 3–42, 2006.
- [17] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016. [Online]. Available: <https://api.semanticscholar.org/CorpusID:4650265>.
- [18] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [19] G. Liu and J. Guo, “Bidirectional lstm with attention mechanism and convolutional layer for text classification,” *Neurocomputing*, vol. 337, pp. 325–338, 2019.
- [20] A. F. Agarap, “Deep learning using rectified linear units (relu),” *arXiv preprint arXiv:1803.08375*, 2018.
- [21] I. Kouretas and V. Paliouras, “Hardware implementation of a softmax-like function for deep learning,” *Technologies*, vol. 8, no. 3, p. 46, 2020.
- [22] D. M. Powers, “Evaluation: From precision, recall and f-measure to roc, informedness, markedness and correlation,” *arXiv preprint arXiv:2010.16061*, 2020.
- [23] I. Düntsch and G. Gediga, “Confusion matrices and rough set data analysis,” in *Journal of Physics: Conference Series*, IOP Publishing, vol. 1229, 2019, p. 012 055.

- [24] B. U. Gaikwad and P. Halkarnikar, "Spam e-mail detection by random forest algorithm," *Int J Adv Comput Eng Commun Technol (IJACECT)*, 2013.
- [25] S. Salloum, T. Gaber, S. Vadera, and K. Shaalan, "A systematic literature review on phishing email detection using natural language processing techniques," *IEEE Access*, vol. 10, pp. 65 703–65 727, 2022.
- [26] F. Alam, P. K. Nath, and M. Khan, "Text to speech for bangla language using festival," BRAC University, Tech. Rep., 2007.
- [27] P. Malhotra and S. Malik, "Spam email detection using machine learning and deep learning techniques," in *Proceedings of the International Conference on Innovative Computing & Communication (ICICC)*, 2022.
- [28] N. Grewal, R. Nijhawan, and A. Mittal, "Email spam detection using machine learning and feature optimization method," in *Distributed Computing and Optimization Techniques: Select Proceedings of ICDCOT 2021*, Springer, 2022, pp. 435–447.
- [29] A. Al Maruf, A. Al Numan, M. M. Haque, T. T. Jidney, and Z. Aung, "Ensemble approach to classify spam sms from bengali text," in *International Conference on Advances in Computing and Data Sciences*, Springer, 2023, pp. 440–453.
- [30] R. Sharma and G. Kaur, "E-mail spam detection using svm and rbf.," *International Journal of Modern Education & Computer Science*, vol. 8, no. 4, 2016.
- [31] H. Qin, "Bilstm text classification incorporating attentional mechanisms," *Academic Journal of Computing & Information Science*, vol. 6, no. 13, pp. 92–98,